

ГЛАВА 15

ОБЪЕКТНО-ОРИЕНТИРОВАННОЕ ПРОГРАММИРОВАНИЕ

В этой главе...

15.1. Краткий обзор ООП	748
15.2. Определение базовых и производных классов	750
15.3. Виртуальные функции	763
15.4. Абстрактные базовые классы	768
15.5. Управление доступом и наследование	772
15.6. Область видимости класса при наследовании	778
15.7. Конструкторы и функции управления копированием	784
15.8. Контейнеры и наследование	795
15.9. Возвращаясь к запросам текста	801
Резюме	817
Термины	817

Объектно-ориентированное программирование основано на трех фундаментальных концепциях: абстракция данных, наследование и динамическое связывание.

Наследование и динамическое связывание рационализируют программы двумя способами: они упрощают создание новых классов, которые подобны, но не идентичны другим классам, а также облегчают написание программы, позволяя игнорировать незначительные различия в подобных классах.

При создании большинства приложений используются одинаковые принципы, которые различаются лишь способами их реализации. Например, рассматриваемый для примера книжный магазин мог бы применять различные системы тарификации для разных книг. Некоторые книги можно было бы продавать лишь по фиксированной цене, а для других применить гибкую систему скидок. Можно было бы предоставлять скидку тем покупателям, которые покупают несколько экземпляров книги. Скидку можно было бы также предоставить на несколько первых экземпляров, а для остальных оставить полную цену. *Объектно-*

ориентированное программирование (Object-Oriented Programming, или ООП) — это наилучший способ создания приложений такого типа.



15.1. Краткий обзор ООП

Ключевыми концепциями объектно-ориентированного программирования являются абстракция данных, наследование и динамическое связывание. Используя абстракцию данных, можно определить классы, отделяющие интерфейс от реализации (см. главу 7). Наследование позволяет определять классы, моделирующие отношения между подобными типами. Динамическое связывание позволяет использовать объекты этих типов, игнорируя незначительные различия между ними.

Наследование

Связанные *наследованием* (inheritance) классы формируют иерархию. В корне иерархии обычно находится *базовый класс* (base class), от которого прямо или косвенно происходят другие классы. Эти унаследованные классы известны как *производные классы* (derived class). В базовом классе определяют те члены, которые будут общими у всех типов в иерархии. В производных классах определяются те члены, которые будут специфическими для данного производного класса.

Для моделирования разных стратегий расценок определим класс `Quote`, который будет базовым классом нашей иерархии. Объект класса `Quote` представит книгу без скидок. От него унаследуем второй класс, `Bulk_quote`, представляющий книги, которые могут быть проданы со скидкой за опт.

У этих классов будут две функции-члена.

- Функция `isbn()` будет возвращать ISBN. Она никак не зависит от специфических особенностей производных классов; поэтому будет определена только в классе `Quote`.
- Функция `net_price(size_t)` будет возвращать цену при покупке определенного количества экземпляров книги. Эта операция специфична для типа; классы `Quote` и `Bulk_quote` определяют собственные версии этой функции.

В языке C++ базовый класс отличает функции, специфические для типа, от тех, которые предполагается наследовать в производных классах без изменений. Те функции, которые производные классы должны определять самостоятельно, базовый класс определяет как `virtual`. Исходя из этого, класс `Quote` можно первоначально написать так:

```
class Quote {  
public:
```

```
std::string isbn() const;
virtual double net_price(std::size_t n) const;
};
```

Производный класс должен указать класс (классы), который он намеревается унаследовать. Для этого используется находящийся после двоеточия *список наследования класса* (class derivation list), представляющий собой разделяемый запятыми список базовых классов, у каждого из которых может быть необязательный спецификатор доступа:

```
class Bulk_quote : public Quote { // Bulk_quote наследуется от Quote
public:
    double net_price(std::size_t) const override;
};
```

Поскольку класс `Bulk_quote` использует в списке наследования спецификатор `public`, его объекты можно использовать так, как *будто* они являются объектами класса `Quote`.

Тело производного класса должно включать объявления всех *виртуальных функций* (virtual function), которые он намеревается определить для себя. Производный класс может включить в эти функции ключевое слово `virtual`, но не обязательно. По причинам, рассматриваемым в разделе 15.3 (стр. 765), новый стандарт позволяет производному классу явно указать, что функция-член предназначена для *переопределения* (override) унаследованной виртуальной функции. Для этого после списка ее параметров располагают ключевое слово `override`.

Динамическое связывание

Динамическое связывание (dynamic binding) позволяет взаимозаменяемо использовать тот же код для обработки объектов как типа `Quote`, так и `Bulk_quote`. Например, следующая функция выводит общую стоимость при покупке заданного количества экземпляров указанной книги:

```
// вычислить и отобразить цену за указанное количество экземпляров
// с применением всех скидок
double print_total(ostream &os,
                  const Quote &item, size_t n)
{
    // в зависимости от типа, связанного с параметром item объекта,
    // вызвать функцию Quote::net_price() или Bulk_quote::net_price()
    double ret = item.net_price(n);
    os << "ISBN: " << item.isbn() // вызов Quote::isbn()
        << " # sold: " << n << " total due: " << ret << endl;
    return ret;
}
```

Эта функция довольно проста — она выводит результаты вызова функций `isbn()` и `net_price()` для своего параметра и возвращает значение, вычисленное вызовом функции `net_price()`.

Однако у этой функции есть два интересных момента: по описанным в разделе 15.2.3 (стр. 759) причинам, поскольку параметр `item` является ссылкой на тип `Quote`, эту функцию можно вызвать как для объекта класса `Quote`, так и для объекта класса `Bulk_quote`. По причинам, описанным в разделе 15.2.1 (стр. 750)), поскольку функция `net_price()` является виртуальной, а функция `print_total()` вызывает ее через ссылку, выполняемая версия функции `net_price()` будет зависеть от типа объекта, переданного функции `print_total()`:

```
// basic имеет тип Quote; bulk имеет тип Bulk_quote
print_total(cout, basic, 20); // вызов версии net_price() класса Quote
print_total(cout, bulk, 20);  // вызов версии net_price()
                             // класса Bulk_quote
```

Первый вызов передает функции `print_total()` объект класса `Quote`. Когда функция `print_total()` вызовет функцию `net_price()`, будет выполнена ее версия из класса `Quote`. В следующем вызове, где аргумент имеет тип `Bulk_quote`, будет выполнена версия функции `net_price()` из класса `Bulk_quote` (применяющая скидку). Поскольку решение о выполняемой версии зависит от типа аргумента, оно может быть принято до времени выполнения. Поэтому динамическое связывание иногда называют *привязкой во время выполнения* (run-time binding).



В языке C++ динамическое связывание происходит тогда, когда обращение к виртуальной функции осуществляется при помощи ссылки (или указателя) на базовый класс.

15.2. Определение базовых и производных классов

Во многих, но не всех случаях базовые и производные классы определяют, как и другие классы, но отличия все же имеются. В этом разделе рассматриваются основные возможности, используемые при определении классов, связанных наследованием.



15.2.1. Определение базового класса

Для начала завершим определение класса `Quote`:

```
class Quote {
public:
    Quote() = default; // = default см. раздел 7.1.4 (стр. 345)
    Quote(const std::string &book, double sales_price):
        bookNo(book), price(sales_price) { }
    std::string isbn() const { return bookNo; }
    // возвращает общую цену за определенное количество проданных
```



```

// экземпляров, а различные системы скидок определяют и
// применяют производные классы
virtual double net_price(std::size_t n) const
{ return n * price; }
virtual ~Quote() = default; // динамическое связывание для
                             // деструктора
private:
    std::string bookNo; // идентификатор экземпляра
protected:
    double price = 0.0; // стандартная цена (без скидки)
};

```

Новым в этом классе являются использование ключевого слова `virtual` в функции `net_price()` и деструкторе, а также спецификатора доступа `protected`. Виртуальные деструкторы рассматриваются в разделе 15.7.1 (стр. 785), а пока следует заметить, что корневой класс иерархии наследования почти всегда определяет виртуальный деструктор.



Базовые классы обычно должны определять виртуальный деструктор. Виртуальные деструкторы необходимы, даже если они не делают ничего.

Функции-члены и наследование

Производные классы наследуют члены своих базовых классов. Но производный класс должен быть в состоянии обеспечить собственное определение таких зависимых от типа операций, как `net_price()`. В таких случаях производный класс должен *переопределить* унаследованное от базового класса определение, обеспечив собственное определение.

В языке C++ базовый класс должен отличать функции, которые предполагается переопределить в производных классах, от тех, которые производные классы, вероятно, наследуют без изменений. Функции, переопределение которых предполагается в производных классах, базовый класс определяет как `virtual`. Когда вызов виртуальной функции происходит *через указатель или ссылку*, он будет привязан динамически. В зависимости от типа объекта, с которым связана ссылка или указатель, будет выполнена версия базового или одного из его производных классов.

Базовый класс определяет, что функция-член должна быть привязана динамически, предвзявляя ее объявление ключевым словом `virtual`. Любая нестатическая функция-член (см. раздел 7.6, стр. 387), кроме конструктора, может быть виртуальной. Ключевое слово `virtual` присутствует только в объявлении в классе и не может использоваться в определении функции вне тела класса. Функция, объявленная виртуальной в базовом классе, неявно является виртуальной и в производных классах. Более подробная информация о виртуальных функциях приведена в разделе 15.3 (стр. 763).

Функции-члены, которые не объявлены как `virtual`, распознаются во время компиляции, а не во время выполнения. Это именно то поведение, которое необходимо для функции `isbn()`. Она не зависит от подробностей производного типа и ведет себя одинаково как с объектами класса `Quote`, так и `Bulk_quote`. В нашей иерархии наследования будет только одна версия функции `isbn()`. Таким образом, не будет никаких вопросов относительно выполняемой версии функции `isbn()` при вызове.

Управление доступом и наследование

Производный класс наследует члены, определенные в его базовом классе. Но функции-члены производного класса не обязаны обращаться к членам, унаследованным от базового класса. Подобно любому другому коду, использующему базовый класс, производный класс может обращаться к открытым членам своего базового класса, но не может обратиться к закрытым членам. Но иногда у базового класса могут быть члены, которые следует позволить использовать в производных классах, но все же запретить доступ к ним другим пользователям. В определении таких членов используется спецификатор доступа `protected`.

Класс `Quote` ожидает, что его производные классы определяют собственную функцию `net_price()`. Для этого им потребуется доступ к члену `price`. В результате класс `Quote` определяет эту переменную-член как `protected`. Производные классы получают доступ к переменной `bookNo` таким же образом, как и обычные пользователи, — при вызове функции `isbn()`. Следовательно, переменная-член `bookNo` останется закрытой и недоступной классам, производным от класса `Quote`. Более подробная информация о защищенных членах приведена в разделе 15.5 (стр. 772).

Упражнения раздела 15.2.1

Упражнение 15.1. Что такое виртуальный член класса?

Упражнение 15.2. Чем спецификатор доступа `protected` отличается от `private`?

Упражнение 15.3. Определите собственные версии класса `Quote` и функции `print_total()`.



15.2.2. Определение производного класса

Производный класс должен определить, от какого класса (классов) он происходит. Для этого используется находящийся после двоеточия *список наследования класса* (`class derivation list`), представляющий собой разделяемый запятыми список имен определенных ранее классов. Каждому имени базового класса может предшествовать необязательный спецификатор доступа: `public`, `protected` или `private`.

Производный класс должен объявить каждую унаследованную функцию-член, которую он намеревается переопределить. Поэтому класс `Bulk_quote` должен включать функцию-член `net_price()`:

```
class Bulk_quote : public Quote { // Bulk_quote происходит от Quote
    Bulk_quote() = default;
    Bulk_quote(const std::string&, double, std::size_t, double);
    // переопределить базовую версию и реализовать политику
    // скидок при оптовых закупках
    double net_price(std::size_t) const override;
private:
    std::size_t min_qty = 0; // минимальная покупка для скидки
    double discount = 0.0; // доля применяемой скидки
};
```

Класс `Bulk_quote` унаследовал функцию `isbn()`, а также переменные-члены `bookNo` и `price` из своего базового класса `Quote`. Он определяет собственную версию функции `net_price()` и имеет две дополнительные переменные-члены — `min_qty` и `discount`, которые определяют минимальное количество экземпляров и скидку, применяемую при его покупке.

Более подробная информация об используемых в списке наследования спецификаторах доступа приведена в разделе 15.5 (стр. 773), а пока достаточно знать, что спецификатор доступа определяет, разрешено ли пользователям производного класса знать, что он унаследован от базового класса.

При открытом наследовании открытые члены базового класса становятся частью интерфейса производного. Кроме того, объект открытого производного типа можно привязать к указателю или ссылке на базовый тип. Поскольку в списке наследования использован спецификатор `public`, интерфейс класса `Bulk_quote` неявно содержит функцию `isbn()`, объект класса `Bulk_quote` можно использовать там, где ожидается указатель или ссылка на объект класса `Quote`.

Большинство классов непосредственно происходит только от одного базового класса. Эта форма наследования, известная как “одинокое наследование”, и является темой данной главы. В разделе 18.3 (стр. 1002) будут описаны классы, у которых в списке наследования больше одного базового класса.

Виртуальные функции в производном классе

Производные классы часто, но не всегда, переопределяют унаследованные виртуальные функции. Если производный класс не переопределяет виртуальную функцию своего базового класса, то, подобно любому другому члену, производный класс наследует версию, определенную в его базовом классе.

**C++
11**

Производный класс может применять к переопределяемым функциям ключевое слово `virtual`, но не обязательно. По причинам, рассматриваемым в разделе 15.3 (стр. 765), новый стандарт позволяет производному классу явно указывать, что функция-член предназначена для пере-

определения унаследованной виртуальной функции. Для этого применяется спецификатор `override` в определении после списка параметров, либо после ключевого слова `const`, либо квалификатора ссылки, если член класса константен (см. раздел 7.1.2, стр. 337), или ссылки на функцию (см. раздел 13.6.3, стр. 691).

Объекты производного класса и преобразование производного в базовый

Объект производного класса состоит из несколькими частями: нестатических членов, определенных в самом производном классе, а также объекта, состоящего из нестатических членов каждого его базового класса, от которых он происходит. Таким образом, объект класса `Bulk_quote` будет содержать четыре части данных: переменные-члены `bookNo` и `price`, унаследованные от класса `Quote`, и переменные-члены `min_qty` и `discount`, определенные в классе `Bulk_quote`.

Хотя стандарт не определяет расположение в памяти производных объектов, объект `Bulk_quote` можно считать состоящим из двух частей (рис. 15.1).

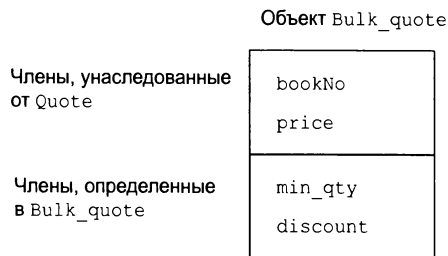


Рис. 15.1. Концептуальная структура объекта класса `Bulk_quote`



Базовые и производные части объекта вовсе не обязательно будут располагаться рядом. Рис. 15.1 — это концептуальное, не физическое представление работы класса.

Поскольку производная часть объекта соответствует его базовому классу (классам), объект производного типа можно использовать так, как *будто* это объект его базового класса (классов). В частности, ссылку или указатель на базовый класс можно связать с частью базового класса производного объекта.

```
Quote item;           // объект базового типа
Bulk_quote bulk;      // объект производного типа
Quote *p = &item;     // p указывает на объект Quote
p = &bulk;            // p указывает на часть bulk объекта Quote
Quote &r = bulk;      // r связан с частью bulk объекта Quote
```

Это преобразование обычно называют *преобразованием производного в базовый* (derived-to-base conversion). Подобно любому другому преобразованию, компилятор применяет его неявно (см. раздел 4.11, стр. 218).

Факт неявного преобразования производного в базовый означает возможность использования объекта производного типа или ссылки на него там, где нужна ссылка на базовый тип. Точно так же можно использовать указатель на производный тип там, где требуется указатель на базовый тип.



Факт наличия в объекте производного класса частей объектов его базовых классов является основой работы наследования.

Конструкторы производного класса

Хотя объект производного класса содержит члены, унаследованные им от базового, он не может инициализировать их непосредственно. Как и любой другой код, создающий объект базового класса, производный класс должен использовать конструктор базового класса для инициализации своей части базового класса.



Каждый класс сам контролирует инициализацию своих членов.

Часть базового класса объекта, наряду с переменными-членами производного класса, инициализируется на этапе инициализации конструктора (см. раздел 7.5.1, стр. 372). Аналогично инициализации переменных-членов, для передачи аргументов конструктору базового класса конструктор производного класса использует свой список инициализации. Рассмотрим конструктор `Bulk_quote()` с четырьмя параметрами:

```
Bulk_quote(const std::string& book, double p,  
           std::size_t qty, double disc) :  
    Quote(book, p), min_qty(qty), discount(disc) { }  
    // как прежде  
};
```

Для инициализации переменных-членов конструктору класса `Quote` передаются его первые два параметра (представляющие ISBN и цену). Этот конструктор инициализирует базовую часть класса `Bulk_quote` (т.е. переменные-члены `bookNo` и `price`). Когда (пустое) тело конструктора класса `Quote` закончит работу, часть базового класса создаваемого объекта будет инициализирована. Затем инициализируются прямые переменные-члены `min_qty` и `discount`. И наконец, выполняется (пустое) тело конструктора класса `Bulk_quote`.

Подобно переменной-члену, если не определено иное, базовая часть производного объекта инициализируется по умолчанию. Чтобы использовать другой конструктор базового класса, следует предоставить список инициа-

ции конструктора, используя имя базового класса, сопровождаемое, как обычно, заключенным в скобки списком аргументов. Эти аргументы используются для выбора конкретного конструктора базового класса для инициализации базовой части объекта производного класса.



Сначала инициализируются члены базового класса, а затем члены производного класса в порядке их объявления.

Использование членов базового класса из производного

Производный класс может обращаться к открытым и защищенным членам своего базового класса:

```
// если приобретено достаточное количество экземпляров,  
// использовать цену со скидкой  
double Bulk_quote::net_price(size_t cnt) const  
{  
    if (cnt >= min_qty)  
        return cnt * (1 - discount) * price;  
    else  
        return cnt * price;  
}
```

Эта функция вычисляет цену со скидкой: если приобретенное количество экземпляров превышает значение переменной `min_qty`, к цене (`price`) применяется скидка (`discount`).

Более подробная информация об областях видимости приведена в разделе 15.6 (стр. 778), а пока достаточно знать, что область видимости производного класса вкладывается в область видимости его базового класса. В результате нет никакого различия между тем, как член производного класса использует члены, определенные в его собственном классе (например, `min_qty` и `discount`), и как он использует члены, определенные в его базовом классе (например, `price`).

КЛЮЧЕВАЯ КОНЦЕПЦИЯ. СОБЛЮДЕНИЕ ИНТЕРФЕЙСА БАЗОВОГО КЛАССА

Важно понимать, что каждый класс определяет собственный интерфейс. Для взаимодействия с объектом типа класса следует использовать интерфейс этого класса, даже если он — часть базового класса в объекте производного.

В результате конструкторы производного класса не могут непосредственно инициализировать члены своего базового класса. Тело конструктора производного класса может присваивать значения его открытым или защищенным членам базового класса. Хотя он *может* присвоить значения этим членам, обычно это *не применяется*. Как и любой другой пользователь базового класса, производный класс должен соблюдать интерфейс своего базового класса, используя для инициализации своих унаследованных членов его конструктор.

Наследование и статические члены

Если в базовом классе определен статический (static) член (см. раздел 7.6, стр. 387), для всей иерархии существует только один его экземпляр. Независимо от количества классов, производных от базового класса, существовать будет только один экземпляр каждого статического члена.

```
class Base {
public:
    static void statmem();
};
class Derived : public Base {
    void f(const Derived&);
};
```

Статические члены подчиняются обычным правилам управления доступом: если член класса объявлен в базовом классе закрытым, производные классы не получают к нему доступа. Когда статический член класса доступен, к нему можно обращаться как из базового, так и из производного класса:

```
void Derived::f(const Derived &derived_obj)
{
    Base::statmem();    // ok: statmem() определена в Base
    Derived::statmem(); // ok: Derived наследует statmem()
    // ok: объект производного класса применим для доступа к
    // статическому члену базового
    derived_obj.statmem(); // доступ в объекте класса Derived
    statmem();            // доступ в объекте этого класса
}
```

Объявления производных классов

Производный класс объявляется как любой другой класс (см. раздел 7.3.3, стр. 362). Объявление содержит имя класса, но не включает его список наследования:

```
class Bulk_quote : public Quote; // ошибка: здесь не может быть списка
                                // наследования
class Bulk_quote;               // ok: правильный способ объявления
                                // производного класса
```

Задача объявления в том, чтобы сообщить о существовании имени и какую сущность он обозначает: класс, функцию или переменную. Список наследования и все другие подробности определения должны присутствовать в теле класса.

Классы, используемые как базовые

Класс должен быть определен, а не только объявлен, прежде чем его можно будет использовать как базовый класс:

```
class Quote; // объявлен, но не определен
// ошибка: класс Quote следует определить
class Bulk_quote : public Quote { ... };
```

Причина этого ограничения очевидна: каждый производный класс содержит и может использовать члены, унаследованные от его базового класса. Чтобы использовать эти члены, производный класс должен знать, что они из себя представляют. Одним из следствий этого правила является невозможность наследования класса от себя самого.

Базовый класс сам может быть производным классом:

```
class Base { /* ... */ };
class D1: public Base { /* ... */ };
class D2: public D1 { /* ... */ };
```

В этой иерархии класс *Base* является *прямым базовым* (direct base class) для класса *D1* и *косвенным базовым* (indirect base class) для класса *D2*. Прямой базовый класс указывают в списке наследования. Косвенный базовый класс наследуется производным через его прямой базовый класс.

Каждый класс наследует все члены своего прямого базового класса. Большинство производных классов наследует члены своего прямого базового класса. Члены прямого базового класса включают унаследованные из его базового класса и т.д. по цепи наследования. Фактически самый последний производный объект содержит часть его прямого базового класса и каждого из его косвенных базовых классов.

Предотвращение наследования

C++11 Иногда определяют класс, от которого не следует получать другие производные классы. Либо может быть определен класс, который не предусматривается как подходящий на роль базового. По новому стандарту можно воспрепятствовать использованию класса как базового, расположив за его именем спецификатор `final`:

```
class NoDerived final { /* */ }; // класс NoDerived не может быть базовым
class Base { /* */ };
// класс Last финальный; нельзя наследовать класс Last
class Last final : Base { /* */ }; // класс Last не может быть базовым
class Bad : NoDerived { /* */ }; // ошибка: класс NoDerived финальный
class Bad2 : Last { /* */ }; // ошибка: класс Last финальный
```

Упражнения раздела 15.2.2

Упражнение 15.4. Какие из следующих объявлений (если они есть) некорректны? Объясните, почему.

```
class Base { ... };
(a) class Derived : public Derived { ... };
```



```
(b) class Derived : private Base { ... };  
(c) class Derived : public Base;
```

Упражнение 15.5. Напишите собственную версию класса `Bulk_quote`.

Упражнение 15.6. Проверьте свою функцию `print_total()` из упражнения раздела 15.2.1 (стр. 752), передав ей объекты класса `Quote` и `Bulk_quote`.

Упражнение 15.7. Определите класс, реализующий ограниченную стратегию скидок, которая применяет скидку только к покупкам до заданного предела. Если количество экземпляров превышает этот предел, к остальным применяется обычная цена.



15.2.3. Преобразования и наследование



Понимание того, как происходит преобразование типов между базовыми и производными классами, очень важно для освоения принципов объектно-ориентированного программирования на языке C++.

Обычно ссылку или указатель можно связать только с тем объектом, тип которого либо совпадает с типом ссылки или указателя (см. раздел 2.3.1, стр. 86 и раздел 2.3.2 стр. 87), либо допускает константное преобразование в него (см. раздел 4.11.2, стр. 222). Классы, связанные наследованием, являются важным исключением: с объектом производного типа можно связать указатель или ссылку на тип базового класса. Например, ссылку `Quote&` можно использовать для обращения к объекту `Bulk_quote`, а адрес объекта `Bulk_quote` можно сохранить в указателе `Quote*`.

У факта возможности привязки ссылки (или указателя) на тип базового класса к объекту производного есть очень важное следствие: при использовании ссылки (или указателя) на тип базового класса неизвестен фактический тип объекта, с которым он связан. Этот объект может быть как объектом базового класса, так и производного.



Подобно встроенным указателям, классы интеллектуальных указателей (см. раздел 12.1, стр. 574) обеспечивают преобразование производного в базовый, позволяя хранить указатель на объект производного типа в интеллектуальном указателе на базовый.



Статический и динамический типы

При использовании связанных наследованием типов нередко приходится отличать *статический тип* (static type) переменной или выражения от *динамического типа* (dynamic type) объекта, который представляет выражение. Статический тип выражения всегда известен на момент компиляции — это тип,

с которым переменная объявляется или возвращает выражение. Динамический тип — это тип объекта в области памяти, которую представляет переменная или выражение. Динамический тип не может быть известен во время выполнения.

Рассмотрим пример, когда функция `print_total()` вызывает функцию `net_price()` (см. раздел 15.1, стр. 749):

```
double ret = item.net_price(n);
```

Известно, что статическим типом параметра `item` является `Quote&`. Динамический тип зависит от типа аргумента, с которым связан параметр `item`. Этот тип не может быть известен, пока не произойдет вызов во время выполнения. Если функции `print_total()` передать объект класса `Bulk_quote`, то статический тип параметра `item` будет отличаться от его динамического типа. Как уже упоминалось, статический тип параметра `item` — это `Quote&`, но в данном случае динамическим типом будет `Bulk_quote`.

Динамический тип выражения, которое не является ни ссылкой, ни указателем, всегда будет совпадать со статическим типом этого выражения. Например, переменная типа `Quote` всегда будет объектом класса `Quote`; нельзя сделать ничего, что изменит тип объекта, которому соответствует эта переменная.



Крайне важно понять, что статический тип указателя или ссылки на базовый класс может отличаться от его динамического типа.

Не существует неявного преобразования из базового типа в производный...

Преобразование из производного в базовый существует благодаря тому, что каждый объект производного класса содержит часть базового класса, с которой и могут быть связаны указатели или ссылки на тип базового класса. Для объектов базового класса подобной гарантии нет. Объект базового класса может существовать либо как независимый объект, либо как часть объекта производного класса. У объекта базового класса, не являющегося частью объекта производного, есть только те члены, которые определены базовым классом; в нем не определены члены производного класса.

Поскольку объект базового класса может быть, а может и не быть частью производного объекта, нет никаких автоматических преобразований из базового класса в класс (классы), производный от него:

```
Quote base;
Bulk_quote* bulkP = &base; // ошибка: нельзя преобразовать базовый в
                           // производный
Bulk_quote& bulkRef = base; // ошибка: нельзя преобразовать базовый в
                           // производный
```

Если бы эти присвоения были допустимы, то можно было бы попытаться использовать указатель `bulkP` или ссылку `bulkRef` для доступа к членам, которые не существуют в объекте `base`.

Немного удивительно то, что невозможно преобразование из базового в производный, даже когда с объектом производного класса связан указатель или ссылка на базовый класс:

```
Bulk_quote bulk;  
Quote *itemP = &bulk;           // ok: динамический тип Bulk_quote  
Bulk_quote *bulkP = itemP;      // ошибка: нельзя преобразовать базовый в  
                                // производный
```

У компилятора нет никакого способа узнать (во время компиляции), что некое преобразование окажется безопасно во время выполнения. Компилятор рассматривает только статические типы указателей или ссылок, определяя допустимость преобразования. Если у базового класса есть одна или несколько виртуальных функций, для запроса преобразования, проверяемого во время выполнения, можно использовать оператор `dynamic_cast` (рассматриваемый в разделе 19.2.1, стр. 1030). В качестве альтернативы, когда известно, что преобразование из базового в производный безопасно, для обхода запрета компилятора можно использовать оператор `static_cast` (см. раздел 4.11.3, стр. 223).



...и нет преобразований между объектами

Автоматическое преобразование производного класса в базовый применимо только для ссылок и указателей. Нет способа преобразования типа производного класса в тип базового класса. Однако нередко вполне возможно преобразовать объект производного класса в тип базового класса. Но такие преобразования не всегда ведут себя так, как хотелось бы.

Помните, что при инициализации или присвоении объекта типа класса фактически происходит вызов функции. При инициализации происходит вызов конструктора (см. раздел 13.1.1, стр. 630 и раздел 13.6.2, стр. 676), а при присвоении — вызов оператора присвоения (см. раздел 13.1.2, стр. 634 и раздел 13.6.2, стр. 679). У этих функций-членов обычно есть параметр, являющийся ссылкой на константную версию типа класса.

Поскольку эти функции-члены получают ссылки, преобразование производного класса в базовый позволяет передавать функциям копирования и перемещения базового класса объект производного класса. Эти функции не являются виртуальными. При передаче объекта производного класса конструктору базового выполняется конструктор, определенный в базовом классе. Этому конструктору известно *только* о членах самого базового класса. Точно так же, если объект производного класса присваивается объекту базового, выполняется оператор присвоения, определенный в базовом классе. Этот оператор также знает *только* о членах самого базового класса.

Например, классы приложения книжного магазина используют синтезируемые версии операторов копирования и присвоения (см. раздел 13.1.1, стр. 631 и раздел 13.1.2, стр. 634). Более подробная информация об управлении копированием и наследовании приведена в разделе 15.7.2 (стр. 786), а пока достаточно знать, что синтезируемые версии осуществляют почленное копирование или присвоение переменных-членов класса тем же способом, что и у любого другого класса:

```
Bulk_quote bulk; // объект производного типа
Quote item(bulk); // используется конструктор Quote::Quote(const Quote&)
item = bulk; // вызов Quote::operator=(const Quote&)
```

При создании объекта `item` выполняется конструктор копий класса `Quote`. Этот конструктор знает только о переменных-членах `bookNo` и `price`. Он копирует эти члены из части `Quote` объекта `bulk` и *игнорирует* члены, являющиеся частью `Bulk_quote` объекта `bulk`. Аналогично при присвоении объекта `bulk` объекту `item` ему присваивается только часть `Quote` объекта `bulk`.

Поскольку часть `Bulk_quote` игнорируется, говорят, что она была *отсечена* (sliced down).



При инициализации объекта базового типа (или присвоении) объектом производного типа копируется, перемещается или присваивается только часть базового класса производного объекта. Производная часть объекта игнорируется.

КЛЮЧЕВАЯ КОНЦЕПЦИЯ. ПРЕОБРАЗОВАНИЯ МЕЖДУ ТИПАМИ, СВЯЗАННЫМИ НАСЛЕДОВАНИЕМ

Есть три правила преобразования связанных наследованием классов, о которых следует помнить.

- Преобразование из производного класса в базовый применимо только к указателю или ссылке.
- Нет неявного преобразования из типа базового класса в тип производного.
- При преобразовании производного в базовый член класса может быть недоступен из-за спецификатора управления доступом. Доступность рассматривается в разделе 15.5 (стр. 774).

Хотя автоматическое преобразование применимо только к указателям и ссылкам, большинство классов в иерархии наследования (явно или неявно) определяют функции-члены управления копированием (см. главу 13). В результате зачастую вполне можно копировать, перемещать и присваивать объекты производного типа объектам базового. Однако копирование, перемещение или присвоение объекта производного типа объекту базового копирует, перемещает или присваивает *только* члены части базового класса объекта.

Упражнения раздела 15.2.3

Упражнение 15.8. Определите статический и динамический типы.

Упражнение 15.9. Когда может возникнуть отличие статического типа выражения от его динамического типа? Приведите три примера, в которых статический и динамический типы отличаются.

Упражнение 15.10. Возвращаясь к обсуждению в разделе 8.1 (стр. 401), объясните, как работает программа на стр. 409, где функции `read()` класса `Sales_data` передавался объект `ifstream`.



15.3. Виртуальные функции

Как уже упоминалось, в языке C++ динамическое связывание происходит при вызове виртуальной функции-члена через ссылку или указатель на тип базового класса (см. раздел 15.1, стр. 749). Поскольку до времени выполнения неизвестно, какая версия функции вызывается, виртуальные функции следует определять *всегда*. Обычно, если функция не используется, ее определение предоставлять необязательно (см. раздел 6.1.2, стр. 273). Однако следует определить каждую виртуальную функцию, независимо от того, будет ли она использована, поскольку у компилятора нет никакого способа определить, используется ли виртуальная функция.

Вызовы виртуальной функции могут быть распознаны во время выполнения

Когда виртуальная функция вызывается через ссылку или указатель, компилятор создает код *распознавания во время выполнения* (decide at run time) вызываемой функции. Вызывается та функция, которая соответствует динамическому типу объекта, связанного с этим указателем или ссылкой.

В качестве примера рассмотрим функцию `print_total()` из раздела 15.1 (стр. 749). Она вызывает функцию `net_price()` своего параметра `item` типа `Quote&`. Поскольку параметр `item` — это ссылка и функция `net_price()` является виртуальной, какая именно из ее версий будет вызвана во время выполнения, зависит от фактического (динамического) типа аргумента, связанного с параметром `item`:

```
Quote base("0-201-82470-1", 50);
print_total(cout, base, 10);    // вызов Quote::net_price()
Bulk_quote derived("0-201-82470-1", 50, 5, .19);
print_total(cout, derived, 10); // вызов Bulk_quote::net_price()
```

В первом вызове параметр `item` связан с объектом типа `Quote`. В результате, когда функция `print_total()` вызовет функцию `net_price()`, выполнится ее

версия, определенная в классе `Quote`. Во втором вызове параметр `item` связан с объектом класса `Bulk_quote`. В этом вызове функция `print_total()` вызывает версию функции `net_price()` класса `Bulk_quote`.

Крайне важно понимать, что динамическое связывание происходит только при вызове виртуальной функции через указатель или ссылку.

```
base = derived;           // копирует часть Quote производного в базовый
base.net_price(20);       // вызов Quote::net_price()
```

Когда происходит вызов виртуальной функции в выражении с обычным типом (не ссылкой и не указателем), такой вызов привязывается во время компиляции. Например, когда происходит вызов функции `net_price()` объекта `base`, нет никаких вопросов о выполняемой версии. Можно изменить значение (т.е. содержимое) объекта, который представляет `base`, но нет никакого способа изменить тип этого объекта. Следовательно, этот вызов распознается во время компиляции как версия `Quote::net_price()`.

Ключевая концепция. Полиморфизм в языке C++

Одной из ключевых концепций ООП является *полиморфизм* (polymorphism). В переводе с греческого языка “полиморфизм” означает множество форм. Связанные наследованием типы считаются полиморфными, поскольку вполне можно использовать многообразие форм этих типов, игнорируя различия между ними. Краеугольным камнем поддержки полиморфизма в языке C++ является тот факт, что статические и динамические типы ссылок и указателей могут отличаться.

Когда при помощи ссылки или указателя на базовый класс происходит вызов функции, определенной в базовом классе, точный тип объекта, для которого будет выполняться функция, неизвестен. Это может быть объект базового класса, а может быть и производного. Если вызываемая функция не виртуальна, независимо от фактического типа объекта, выполнена будет та версия функции, которая определена в базовом классе. Если функция виртуальна, решение о фактически выполняемой версии функции откладывается до времени выполнения. Она определяется на основании типа объекта, с которым связана ссылка или указатель.

С другой стороны, вызовы неvirtуальных функций связываются во время компиляции. Точно так же вызовы любой функции (виртуальной или нет) для объекта связываются во время компиляции. Тип объекта фиксирован и неизменен — никак нельзя заставить динамический тип объекта отличаться от его статического типа. Поэтому вызовы для объекта связываются во время компиляции с версией, определенной типом объекта.



Виртуальные функции распознаются во время выполнения, *только если* вызов осуществляется через ссылку или указатель. Только в этих случаях динамический тип объекта может отличаться от его статического типа.

Виртуальные функции в производном классе

При переопределении виртуальной функции производный класс может, но не обязан, повторить ключевое слово `virtual`. Как только функция объявляется виртуальной, она остается виртуальной во всех производных классах.

У функции производного класса, переопределяющей унаследованную виртуальную функцию, должны быть точно такие же типы параметров, как и у функции базового класса, которую она переопределяет.

За одним исключением тип возвращаемого значения виртуальной функции в производном классе также должен соответствовать типу возвращаемого значения функции в базовом классе. Исключение относится к виртуальным функциям, возвращающим ссылку (или указатель) на тип, который сам связан наследованием. Таким образом, если тип `D` происходит от типа `B`, то виртуальная функция базового класса может вернуть указатель на тип `B*`, а ее версия в производном классе может вернуть указатель на тип `D*`. Но такие типы возвращаемого значения требуют, чтобы преобразование производного класса в базовый из типа `D` в тип `B` было доступно. Доступность базового класса рассматривается в разделе 15.5 (стр. 774). Пример такого вида виртуальной функции рассматривается в разделе 15.8.1 (стр. 799).



Функция, являющаяся виртуальной в базовом классе, неявно остается виртуальной в его производных классах. Когда производный класс переопределяет виртуальную функцию, ее параметры в базовом и производных классах должны точно совпадать.

Спецификаторы `final` и `override`

Как будет продемонстрировано в разделе 15.6 (стр. 782), производный класс вполне может определить функцию с тем же именем, что и виртуальная функция в его базовом классе, но с другим списком параметров. Компилятор полагает, что такая функция независима от функции базового класса. В таких случаях версия в производном классе не переопределяет версию в базовом. На практике такие объявления зачастую являются ошибкой — автор класса намеревался переопределить виртуальную функцию базового класса, но сделал ошибку в определении списка параметров.



Поиск таких ошибок может быть на удивление трудным. По новому стандарту можно задать переопределение виртуальной функции в производном классе. Это дает ясно понять наше намерение и (что еще более важ-

но) позволяет компилятору самому находить такие проблемы. Компилятор отвергнет программу, если функция, отмеченная как `override`, не переопределит существующую виртуальную функцию:

```
struct B {
    virtual void f1(int) const;
    virtual void f2();
    void f3();
};
struct D1 : B {
    void f1(int) const override; // ok: f1() соответствует f1() базового
    void f2(int) override;      // ошибка: B не имеет функции f2(int)
    void f3() override;        // ошибка: f3() не виртуальная функция
    void f4() override;        // ошибка: B не имеет функции f4()
};
```

В структуре `D1` спецификатор `override` для функции `f1()` вполне подходит; и базовые, и производные версии функции-члена `f1()` константы, они получают тип `int` и возвращают `void`. Версия `f1()` в структуре `D1` правильно переопределяет виртуальную функцию, которую она унаследовала от структуры `B`.

Объявление функции `f2()` в структуре `D1` не соответствует объявлению функции `f2()` в структуре `B` — она не получает никаких аргументов, а определенная в структуре `D1` получает аргумент типа `int`. Поскольку объявления не совпадают, функция `f2()` в структуре `D1` не переопределяет функцию `f2()` структуры `B`; это новая функция со случайно совпавшим именем. Как уже упоминалось, это объявление должно было быть переопределено, но этого не произошло и компилятор сообщил об ошибке.

Поскольку переопределена может быть только виртуальная функция, компилятор отвергнет также функцию `f3()` в структуре `D1`. Эта функция не виртуальна в структуре `B`, поэтому нечего и переопределять.

Точно так же ошибочна и функция `f4()`, поскольку в структуре `B` даже нет такой функции.

Функцию можно также определить как `final`. Любая попытка переопределения функции, которая была определена со спецификатором `final`, будет помечена как ошибка:

```
struct D2 : B {
    // наследует f2() и f3() из B и переопределяет f1(int)
    void f1(int) const final; // последующие классы не могут
                             // переопределять f1(int)
};
struct D3 : D2 {
    void f2(); // ok: переопределение f2() унаследованной от косвенно
               // базовой структуры B
    void f1(int) const; // ошибка: D2 объявила f2() как final
};
```


Спецификаторы `final` и `override` располагаются после списка параметров (включая квалификаторы ссылки или `const`) и после замыкающего типа (см. раздел 6.3.3, стр. 301).

Виртуальные функции и аргументы по умолчанию

Подобно любой другой функции, виртуальная функция может иметь аргументы по умолчанию (см. раздел 6.5.1, стр. 308). Если вызов использует аргумент по умолчанию, то используемое значение определяется статическим типом, для которого вызвана функция.

Таким образом, при вызове через ссылку или указатель на базовый класс аргумент (аргументы) по умолчанию будет определен в базовом классе. Аргументы базового класса будут использоваться даже тогда, когда выполняется версия функции производного класса. В данном случае функции производного класса будут переданы аргументы по умолчанию, определенные для версии функции базового класса. Если функция производного класса будет полагаться на передачу других аргументов, то программа не будет выполняться, как ожидалось.



Виртуальные функции с аргументами по умолчанию должны использовать те же значения аргументов в базовом и производных классах.

Хитрость виртуального механизма

В некоторых случаях необходимо предотвратить динамическое связывание вызова виртуальной функции; нужно вынудить вызов использовать конкретную версию этой виртуальной функции. Для этого используется оператор области видимости. Рассмотрим, например, этот код:

```
// вызов версии базового класса независимо от динамического типа baseP
double undiscounted = baseP->Quote::net_price(42);
```

Здесь происходит вызов версии функции `net_price()` класса `Quote` независимо от типа объекта, на который фактически указывает `baseP`. Этот вызов будет распознан во время компиляции.



Обычно только код функций-членов (или друзей) должен использовать оператор области видимости для обхода виртуального механизма.

Зачем обходить виртуальный механизм? Наиболее распространен случай, когда виртуальная функция производного класса вызывает версию базового класса. В таких случаях версия базового класса могла бы выполнять действия, общие для всей иерархии типов. Версии, определенные в производных классах, осуществляли бы любые дополнительные действия, специфичные для их собственного типа.



Если виртуальная функция производного класса, намереваясь вызвать свою версию из базового класса, пропустит оператор области видимости, то вызов будет распознан во время выполнения как вызов самой версии производного класса, что приведет к бесконечной рекурсии.

Упражнения раздела 15.3

Упражнение 15.11. Добавьте в иерархию класса `Quote` виртуальную функцию `debug()`, отображающую переменные-члены соответствующих классов.

Упражнение 15.12. Возможен ли случай, когда полезно объявить функцию-член и как `override`, и как `final`? Объясните, почему.

Упражнение 15.13. С учетом следующих классов объясните каждую из функций `print()`:

```
class base {
public:
    string name() { return basename; }
    virtual void print(ostream &os) { os << basename; }
private:
    string basename;
};
class derived : public base {
public:
    void print(ostream &os) { print(os); os << " " << i; }
private:
    int i;
};
```

Если в этом коде имеются ошибки, устранили их.

Упражнение 15.14. С учетом классов из предыдущего упражнения и следующих объектов укажите, какие из версий функций будут применены во время выполнения:

```
base bobj;           base *bp1 = &bobj;   base &br1 = bobj;
derived dobj;        base *bp2 = &dobj;   base &br2 = dobj;
(a) bobj.print();    (b) dobj.print();    (c) bp1->name();
(d) bp2->name();     (e) br1.print();     (f) br2.print();
```

15.4. Абстрактные базовые классы

Предположим, что классы приложения книжного магазина необходимо дополнить поддержкой нескольких стратегий скидок. Кроме оптовой скидки, можно было бы предоставить скидку за покупку до определенного количества, а свыше применять полную цену. Либо можно было бы предоставить скидку за покупку свыше одного предела, но не выше другого.

Для всех этих стратегий необходимы одинаковые средства: количество экземпляров и объем скидки. Для поддержки этих столь разных стратегий можно определить новый класс по имени `Disc_quote`, позволяющий хранить количество экземпляров и объем скидки. Такие классы как `Bulk_item`, предоставляющие определенную стратегию скидок, наследуются от класса `Disc_quote`. Каждый из производных классов реализует собственную стратегию скидок, определяя собственную версию функции `net_price()`.

Прежде чем определять собственный класс `Disc_Quote`, следует решить, что будет делать функция `net_price()`. Класс `Disc_quote` не будет соответствовать никакой конкретной стратегии скидок; для этого класса нет никакого смысла создавать функцию `net_price()`.

Класс `Disc_quote` можно было бы определить без его собственной версии функции `net_price()`. В данном случае класс `Disc_quote` наследовал бы функцию `net_price()` от класса `Quote`.

Однако такой проект позволил бы пользователям писать бессмысленный код. Пользователь мог бы создать объект типа `Disc_quote`, предоставив количество и объем скидки. Передача объекта класса `Disc_quote` такой функции, как `print_total()`, задействовала бы версию функции `net_price()` из класса `Quote`. Вычисляемая цена не включила бы скидку, предоставляемую при создании объекта. Такое поведение не имеет никакого смысла.

Чистые виртуальные функции

Тщательный анализ этого вопроса показывает, что проблема не только в том, что неизвестно, как определить функцию `net_price()`. Практически следовало бы запретить пользователям создавать объекты класса `Disc_quote` вообще. Этот класс представляет общую концепцию скидки на книги, а не конкретную стратегию скидок.

Для воплощения этого намерения (и однозначного уведомления о бессмысленности функции `net_price()`) определим функцию `net_price()` как *чистую виртуальную функцию* (pure virtual). В отличие от обычных виртуальных функций, чистая виртуальная функция не должна быть определена. Для определения виртуальной функции как чистой вместо ее тела используется часть `= 0` (т.е. как раз перед точкой с запятой, завершающей объявление). Часть `= 0` может присутствовать только в объявлении виртуальной функции в теле класса:

```
// класс для содержания объема скидки и количества экземпляров
// используя эти данные, производные классы реализуют стратегии скидок
class Disc_quote : public Quote {
public:
    Disc_quote() = default;
    Disc_quote(const std::string& book, double price,
               std::size_t qty, double disc):
```

```

        Quote(book, price),
        quantity(qty), discount(disc) { }
    double net_price(std::size_t) const = 0;
protected:
    std::size_t quantity = 0; // минимальная покупка для скидки
    double discount = 0.0;    // доля применяемой скидки
};

```

Подобно прежнему классу `Bulk_item`, класс `Disc_quote` определяет стандартный конструктор и конструктор, получающий четыре параметра. Хотя объекты этого типа нельзя создавать непосредственно, конструкторы в классах, производных от класса `Disc_quote`, будут использовать конструкторы `Disc_quote()` для построения части `Disc_quote` своих объектов. Конструктор с четырьмя параметрами передает первые два конструктору `Quote()`, а двумя последними непосредственно инициализирует собственные переменные-члены `discount` и `quantity`. Стандартный конструктор инициализирует эти члены значениями по умолчанию.

Следует заметить, что определение для чистой виртуальной функции предоставить нельзя. Однако тело функции следует определить вне класса. Поэтому нельзя предоставить в классе тело функции, для которой использована часть = 0.

Классы с чистыми виртуальными функциями являются абстрактными

Класс, содержащий (или унаследовавший без переопределения) чистую виртуальную функцию, является *абстрактным классом* (abstract base class). Абстрактный класс определяет интерфейс для переопределения последующими классами. Нельзя (непосредственно) создавать объекты абстрактного класса. Поскольку класс `Disc_quote` определяет функцию `net_price()` как чистую виртуальную, нельзя определить объекты типа `Disc_quote`. Можно определить объекты классов, производных от `Disc_quote`, если они переопределят функцию `net_price()`:

```

// Disc_quote объявляет чистые виртуальные функции, которые
// переопределяет Bulk_quote
Disc_quote discounted; // ошибка: нельзя определить объект Disc_quote
Bulk_quote bulk;      // ok: у Bulk_quote нет чистых виртуальных функций

```

Классы, унаследованные от класса `Disc_quote`, должны определить функцию `net_price()`, иначе они также будут абстрактными.



Нельзя создать объекты абстрактного класса.

Конструктор производного класса инициализирует только свой прямой базовый класс

Теперь можно повторно реализовать класс `Bulk_quote` так, чтобы он происходил от класса `Disc_quote`, а не непосредственно от класса `Quote`:

```
// скидка прекращается при продаже определенного количества экземпляров
// скидка выражается как доля сокращения полной цены
class Bulk_quote : public Disc_quote {
public:
    Bulk_quote() = default;
    Bulk_quote(const std::string& book, double price,
               std::size_t qty, double disc):
        Disc_quote(book, price, qty, disc) { }
    // переопределение базовой версии для реализации политики скидок
    double net_price(std::size_t) const override;
};
```

У этой версии класса `Bulk_quote` есть *прямой базовый класс* (direct base class), `Disc_quote`, и *косвенный базовый класс* (indirect base class), `Quote`. У каждого объекта класса `Bulk_quote` есть три внутренних объекта: часть `Bulk_quote` (пустая), часть `Disc_quote` и часть `Quote`.

Как уже упоминалось, каждый класс контролирует инициализацию объектов своего типа. Поэтому, даже при том, что у класса `Bulk_quote` нет собственных переменных-членов, он предоставляет тот же конструктор на четыре аргумента, что и первоначальный класс. Новый конструктор передает свои аргументы конструктору класса `Disc_quote`. Этот конструктор, в свою очередь, запускает конструктор `Quote()`. Конструктор `Quote()` инициализирует переменные-члены `bookNo` и `price` объекта `bulk`. Когда конструктор `Quote()` завершает работу, начинает работу конструктор `Disc_quote()`, инициализирующий переменные-члены `quantity` и `discount`. Теперь возобновляет работу конструктор `Bulk_quote()`. Он не делает ничего и ничего не инициализирует.

Ключевая концепция. Рефакторинг

Добавление класса `Disc_quote` в иерархию `Quote` является примером *рефакторинга* (refactoring). Рефакторинг подразумевает переделку иерархии классов с передачей некоторых функций и/или данных из одного класса в другой. Рефакторинг весьма распространен в объектно-ориентированных приложениях.

Примечательно, что, несмотря на изменение иерархии наследования, код, который использует классы `Bulk_quote` и `Quote`, изменять не придется. Но после рефакторинга классов (или любых других изменений) следует перекомпилировать весь код, который использует эти классы.

Упражнения раздела 15.4

Упражнение 15.15. Определите собственные версии классов `Disc_quote` и `Bulk_quote`.

Упражнение 15.16. Перепишите класс из упражнения 15.2.2 раздела 12.1.6 (стр. 758), представляющий ограниченную стратегию скидок, так, чтобы он происходил от класса `Disc_quote`.

Упражнение 15.17. Попытайтесь определить объект типа `Disc_quote` и посмотрите, какие сообщения об ошибке выдал компилятор.



15.5. Управление доступом и наследование

Подобно тому, как каждый класс контролирует инициализацию своих переменных-членов (см. раздел 15.2.2, стр. 755), каждый класс контролирует также *доступность* (accessible) своих членов для производного класса.

Защищенные члены

Как уже упоминалось, класс использует защищенные члены в тех случаях, когда желает предоставить к ним доступ из производных классов, но защитить их от общего доступа. Спецификатор доступа `protected` можно считать гибридом спецификаторов `private` и `public`.

- Подобно закрытым, защищенные члены недоступны пользователям класса.
- Подобно открытым, защищенные члены доступны для членов и друзей классов, производных от данного класса.

Кроме того, защищенный член имеет еще одно важное свойство.

- Производный член класса или дружественный класс может обратиться к защищенным членам базового класса *только* через объект производного. У производного класса нет никакого специального способа доступа к защищенным членам объектов базового класса.

Чтобы лучше понять это последнее правило, рассмотрим следующий пример:

```
class Base {
protected:
    int prot_mem; // защищенный член
};
class Sneaky : public Base {
    friend void clobber(Sneaky&); // есть доступ к Sneaky::prot_mem
    friend void clobber(Base&);   // нет доступа к Base::prot_mem
    int j;                       // j по умолчанию закрытая
};
// ok: clobber может обращаться к закрытым и защищенным членам Sneaky
```

```
void clobber(Sneaky &s) { s.j = s.prot_mem = 0; }
// ошибка: clobber не может обращаться к защищенным членам Base
void clobber(Base &b) { b.prot_mem = 0; }
```

Если производные классы (и друзья) смогут обращаться к защищенным членам в объекте базового класса, то вторая версия функции `clobber` (получающая тип `Base&`) будет корректна. Хотя эта функция и не дружественна классу `Base`, она все же сможет изменить объект типа `Base`; для обхода защиты спецификатором `protected` любого класса достаточно определить новый класс по линии `Sneaky`.

Для предотвращения такого способа применения члены и друзья производного класса могут обращаться к защищенным членам *только* тех объектов базового класса, которые встроены в объект производного; к обычным объектам базового типа у них никакого доступа нет.

Открытое, закрытое и защищенное наследование

Доступ к члену наследуемого класса контролируется комбинацией спецификатора доступа этого члена в базовом классе и спецификатором доступа в списке наследования производного класса. Для примера рассмотрим следующую иерархию:

```
class Base {
public:
    void pub_mem(); // открытый член
protected:
    int prot_mem;   // защищенный член
private:
    char priv_mem; // закрытый член
};
struct Pub_Derv : public Base {
    // ok: производный класс имеет доступ к защищенным членам
    int f() { return prot_mem; }
    // ошибка: закрытые члены недоступны производным классам
    char g() { return priv_mem; }
};
struct Priv_Derv : private Base {
    // закрытое наследование не затрагивает доступ в производном классе
    int fl() const { return prot_mem; }
};
```

Спецификатор доступа наследования никак не влияет на возможность членов (и друзей) производного класса обратиться к членам его собственного прямого базового класса. Доступ к членам базового класса контролируется спецификаторами доступа в самом базовом классе. Структуры `Pub_Derv` и `Priv_Derv` могут обращаться к защищенному члену `prot_mem`, но ни одна из них не может обратиться к закрытому члену `priv_mem`.

Задача спецификатора доступа наследования — контролировать доступ пользователей производного класса, включая другие классы, производные от него, к членам, унаследованным от класса `Base`:

```
Pub_Derv d1; // члены, унаследованные от Base, являются открытыми
Priv_Derv d2; // члены, унаследованные от Base, являются закрытыми
d1.pub_mem(); // ok: pub_mem является открытой в производном классе
d2.pub_mem(); // ошибка: pub_mem является закрытой в производном классе
```

Структуры `Pub_Derv` и `Priv_Derv` унаследовали функцию `pub_mem()`. При открытом наследовании члены сохраняют свой спецификатор доступа. Таким образом, объект `d1` может вызвать функцию `pub_mem()`. В структуре `Priv_Derv` члены класса `Base` являются закрытыми; пользователи этого класса не смогут вызвать функцию `pub_mem()`.

Спецификатор доступа наследования, используемый производным классом, также контролирует доступ из классов, унаследованных от этого производного класса:

```
struct Derived_from_Public : public Pub_Derv {
    // ok: Base::prot_mem остается защищенной в Pub_Derv
    int use_base() { return prot_mem; }
};

struct Derived_from_Private : public Priv_Derv {
    // ошибка: Base::prot_mem является закрытой в Priv_Derv
    int use_base() { return prot_mem; }
};
```

Классы, производные от структуры `Pub_Derv`, могут обращаться к переменной-члену `prot_mem` класса `Base`, поскольку она остается защищенным членом в структуре `Pub_Derv`. У классов, производных от структуры `Priv_Derv`, напротив, такого доступа нет. Все члены, которые структура `Priv_Derv` унаследовала от класса `Base`, являются закрытыми.

Если бы был определен другой класс, скажем, `Prot_Derv`, использующий защищенное наследование, открытые члены класса `Base` в этом классе будут защищенными. У пользователей структуры `Prot_Derv` не было бы никакого доступа к функции `pub_mem()`, но ее члены и друзья могли бы обратиться к унаследованному члену.



Доступность преобразования производного класса в базовый класс

Будет ли доступно преобразование производного класса в базовый класс (см. раздел 15.2.2, стр. 754), зависит от того, какой код пытается использовать преобразование, а также от спецификатора доступа, используемого при наследовании производного класса. С учетом, что класс `D` происходит от класса `B`:

- Пользовательский код может использовать преобразование производного класса в базовый, *только если* класс D открыто наследует класс B. Пользовательский код не может использовать преобразование, если наследование было защищенным или закрытым.
- Функции-члены и друзья класса D могут использовать преобразование в B независимо от вида наследования D от B. Преобразование производного в прямой базовый класс всегда доступно для членов и друзей производного класса.
- Функции-члены и друзья классов, производных от класса D, могут использовать преобразование производного класса в базовый, если наследование было открытым или защищенным. Такой код не сможет использовать преобразование, если наследование классом D класса B было закрытым.



В любом месте кода, где доступен открытый член базового класса, будет доступно также преобразование производного класса в базовый, но не наоборот.

Ключевая концепция. Проект класса и защищенные члены

Без наследования у класса будет два разных вида пользователей: обычные пользователи и *разработчики* (implementor). Обычные пользователи пишут код, который использует объекты типа класса; такой код может обращаться только к открытым членам класса (интерфейсу). Разработчики пишут код, содержащийся в членах и друзьях класса. Члены и друзья класса могут обращаться и к открытым, и к закрытым разделам (реализации).

При наследовании появляется третий вид пользователей, а именно производные классы. Базовый класс делает защищенными те части своей реализации, которые позволено использовать его производным классам. Защищенные члены остаются недоступными обычному пользовательскому коду; закрытые члены остаются недоступными производным классам и их друзьям.

Подобно любому другому классу, базовый класс объявляет члены своего интерфейса открытыми. Класс, используемый как базовый, может разделить свою реализацию на члены, доступные для производных классов и доступные только для базового класса и его друзей. Член класса, относящийся к реализации, должен быть защищен, если он предоставляет функцию или данные, которые производный класс должен будет использовать в собственной реализации. В противном случае члены реализации должны быть закрытыми.

Дружественные отношения и наследование

Подобно тому, как дружественные отношения не передаются (см. раздел 7.3.4, стр. 363), они также не наследуются. У друзей базового класса нет никаких специальных прав доступа к членам его производных классов, а у друзей производного класса нет специальных прав доступа к базовому классу:

```
class Base {
    // добавлено объявление; другие члены, как прежде
    friend class Pal; // у Pal нет доступа к классам, производным от Base
};
class Pal {
public:
    int f(Base b) { return b.prot_mem; } // ok: Pal дружен Base
    int f2(Sneaky s) { return s.j; }      // ошибка: Pal не
                                         // дружен Sneaky
    // доступ к базовому классу контролируется базовым классом, даже в
    // объекте производного
    int f3(Sneaky s) { return s.prot_mem; } // ok: Pal дружен
};
```

Факт допустимости функции `f3()` может показаться удивительным, но он непосредственно следует из правила, что все классы контролируют доступ к собственным членам. Класс `Pal` — друг класса `Base`, поэтому класс `Pal` может обращаться к членам объектов класса `Base`. Это относится и к встроенным в объект класса `Base` объектам классов, производных от него.

Когда класс объявляет другой класс дружественным, это относится только к данному классу, ни его базовые, ни производные классы никаких специальных прав доступа не имеют:

```
// у D2 нет доступа к закрытым или защищенным членам Base
class D2 : public Pal {
public:
    int mem(Base b)
        { return b.prot_mem; } // ошибка: дружба не наследуется
};
```



Дружественные отношения не наследуются; каждый класс сам контролирует доступ к своим членам.

Освобождение индивидуальных членов

Иногда необходимо изменить уровень доступа к имени, унаследованному производным классом. Для этого можно использовать объявление `using` (см. раздел 3.1, стр. 124):

```
class Base {
public:
    std::size_t size() const { return n; }
protected:
    std::size_t n;
```

```
};
class Derived : private Base { // заметьте, наследование закрытое
public:
    // обеспечить уровни доступа для членов, связанных с размером объекта
    using Base::size;
protected:
    using Base::n;
};
```

Поскольку класс `Derived` использует закрытое наследование, унаследованные члены `size()` и `n` по умолчанию будут закрытыми членами класса `Derived`. Объявления `using` корректируют доступность этих членов. Пользователи класса `Derived` могут обращаться к функции-члену `size()`, а классы, впоследствии произошедшие от класса `Derived`, смогут обратиться к переменной `n`.

Объявление `using` в классе может использовать имя любого доступного (не закрытого) члена прямого или косвенного базового класса. Доступность имени, указанного в объявлении `using`, зависит от спецификатора доступа, предшествующего объявлению `using`. Таким образом, если объявление `using` расположено в разделе `private` класса, то имя будет доступно только для членов и друзей. Если объявление находится в разделе `public`, имя доступно для всех пользователей класса. Если объявление находится в разделе `protected`, имя доступно только для членов, друзей и производных классов.



Производный класс может предоставить объявление `using` только для тех имен, доступ к которым разрешен.

Уровни защиты наследования по умолчанию

В разделе 7.2 (стр. 350) упоминалось о том, что у классов, определенных с использованием ключевых слов `struct`, и `class` разные спецификаторы доступа по умолчанию. Точно так же заданный по умолчанию спецификатор наследования зависит от ключевого слова, используемого при определении производного класса. По умолчанию у производного класса, определенного с ключевым словом `class`, будет *закрытое наследование* (`private inheritance`), а с ключевым словом `struct` — *открытое* (`public inheritance`):

```
class Base { /* ... */ };
struct D1 : Base { /* ... */ }; // открытое наследование по умолчанию
class D2 : Base { /* ... */ }; // закрытое наследование по умолчанию
```

Весьма распространено заблуждение, что между классами и структурами есть иные, более глубокие различия. Единственное различие — заданные по умолчанию спецификаторы доступа для членов и наследования. Никаких других различий нет.



Для закрытого наследования производный класс должен быть явно определен как `private`, не следует полагаться на поведение по умолчанию. Это ясно дает понять, что закрытое наследование применено преднамеренно, а не по оплошности.

Упражнения раздела 15.5

Упражнение 15.18. С учетом классов со стр. 773 и 774 и типов объектов, приведенных в комментариях, укажите, какие из следующих присвоений допустимы. Объясните, почему некорректны недопустимые.

```
Base *p = &d1; // d1 имеет тип Pub_Derv
p = &d2;      // d2 имеет тип Priv_Derv
p = &d3;      // d3 имеет тип Prot_Derv
p = &dd1;     // dd1 имеет тип Derived_from_Public
p = &dd2;     // dd2 имеет тип Derived_from_Private
p = &dd3;     // dd3 имеет тип Derived_from_Protected
```

Упражнение 15.19. Предположим, у каждого из классов со стр. 773 и 774 есть функция-член в формате

```
void memfcn(Base &b) { b = *this; }
```

Укажите, была ли эта функция допустима для каждого класса.

Упражнение 15.20. Напишите код проверки ответов на предыдущие два упражнения.

Упражнение 15.21. Выберите одну из следующих общих абстракций, содержащих семейство типов (или любую собственную). Организуйте типы в иерархию наследования.

- (а) Форматы графических файлов (например: gif, tiff, jpeg, bmp)
- (б) Геометрические примитивы (например: box, circle, sphere, cone)
- (в) Типы языка C++ (например: class, function, member function)

Упражнение 15.22. Укажите имена некоторых из наиболее вероятных виртуальных функций, а также открытых и защищенных членов для класса, выбранного в предыдущем упражнении.



15.6. Область видимости класса при наследовании

Каждый класс определяет собственную *область видимости* (scope) (см. раздел 7.4, стр. 366), в рамках которой определены его члены. При наследовании область видимости производного класса (см. раздел 2.2.4, стр. 82) вкладывается в области видимости его базовых классов. Если имя не найдено в области видимости производного класса, поиск его определения продолжается в областях видимости базовых классов.

Тот факт, что область видимости производного класса вложена в область видимости его базовых классов, может быть удивителен. В конце концов, базовые и производные классы определяются в разных частях текста программы. Но именно это иерархическое вложение областей видимости класса позволяет членам производного класса использовать члены его базового класса, как будто они являются частью производного класса. Рассмотрим пример:

```
Bulk_quote bulk;
cout << bulk.isbn();
```

В этом коде поиск определения имени `isbn()` осуществляется следующим образом.

- Поскольку вызывается функция `isbn()` объекта типа `Bulk_quote`, поиск начинается в классе `Bulk_quote`. В этом классе имя `isbn()` не найдено.
- Поскольку класс `Bulk_quote` происходит от класса `Disc_quote`, в нем и продолжается поиск. Имя все еще не найдено.
- Поскольку класс `Disc_quote` происходит от класса `Quote`, поиск продолжается в нем. В этом классе находится определение имени `isbn()`; таким образом, вызов `isbn()` распознается как вызов функции `isbn()` класса `Quote`.

Поиск имен осуществляется во время компиляции

Статический тип (см. раздел 15.2.3, стр. 759) объекта, ссылки или указателя определяет, какие члены этого объекта будут видимы. Даже когда статический и динамический типы отличаются (это бывает в случае, когда используется ссылка или указатель на базовый класс), именно статический тип определяет применимые члены. Например, в класс `Disc_quote` можно было бы добавить функцию-член, которая возвращает пару (тип `pair`) (см. раздел 11.2.3, стр. 545), содержащую минимальное (или максимальное) количество и цену со скидкой.

```
class Disc_quote : public Quote {
public:
    std::pair<size_t, double> discount_policy() const
    { return {quantity, discount}; }
    // другие члены как прежде
};
```

Функцию `discount_policy()` можно использовать только через объект, указатель, или ссылку на тип `Disc_quote`, или класс, производный от него:

```
Bulk_quote bulk;
Bulk_quote *bulkP = &bulk; // статический и динамический типы совпадают
Quote *itemP = &bulk;      // статический и динамический типы отличаются
bulkP->discount_policy();   // ok: bulkP имеет тип Bulk_quote*
itemP->discount_policy();   // ошибка: itemP имеет тип Quote*
```

Хотя объект `bulk` имеет функцию-член `discount_policy()`, она недоступна через указатель `itemP`. Тип `itemP` — указатель на тип `Quote`, а это значит, что поиск имени `discount_policy()` начнется в классе `Quote`. У класса `Quote` нет члена по имени `discount_policy()`, поэтому вызов этой функции-члена объекта, ссылки или указателя на тип `Quote` невозможен.

Конфликт имен и наследование

Как и любая другая, область видимости производного класса позволяет повторно использовать имя, определенное в его прямом или косвенном базовом классе. Как обычно, имена, определенные во внутренней области видимости (например, в производном классе), скрывают имена во внешней области видимости (например, в базовом классе) (см. раздел 2.2.4, стр. 82):

```
struct Base {
    Base(): mem(0) { }
protected:
    int mem;
};
struct Derived : Base {
    Derived(int i): mem(i) { } // Derived::mem инициализируется i
                               // Base::mem инициализируется по умолчанию
    int get_mem() { return mem; } // возвращает Derived::mem
protected:
    int mem; // скрывает mem в Base
};
```

Ссылка на переменную `mem` в функции `get_mem()` распознается как имя в классе `Derived`. Таким образом, код

```
Derived d(42);
cout << d.get_mem() << endl; // выводит 42
```

выведет значение 42.



Член производного класса, имя которого совпадает с именем члена базового класса, скрывает член базового класса и предотвращает прямой доступ к нему.

Применение оператора области видимости для доступа к скрытым членам

Для доступа к скрытому члену базового класса можно использовать оператор области видимости.

```
struct Derived : Base {
    int get_base_mem() { return Base::mem; }
    // ...
};
```

Оператор области видимости изменяет нормальный порядок поиска и заставляет компилятор начинать поиск имени `mem` с класса `Base`. Если бы код выше был выполнен с этой версией класса `Derived`, то результатом вызова `d.get_mem()` был бы 0.

Рекомендуем

Кроме переопределения унаследованных виртуальных функций, производный класс обычно не должен повторно использовать имена, определенные в его базовом классе.

КЛЮЧЕВАЯ КОНЦЕПЦИЯ. ПОИСК ИМЕНИ И НАСЛЕДОВАНИЕ

Для понимания наследования в языке C++ крайне важно знать, как распознаются вызовы функций. Процесс распознавания вызова `p->mem()` (или `obj.mem()`) проходит в четыре этапа.

- Сначала определяется статический тип объекта `p` (или `obj`). Поскольку это вызов члена класса, тип будет классом.
- Поиск имени `mem` осуществляется в классе, который соответствует статическому типу объекта `p` (или `obj`). Если функция `mem()` не найдена, поиск продолжается в прямом базовом классе и далее по цепи классов, пока имя `mem` не будет найдено или пока не будет осмотрен последний класс. Если функция `mem()` не будет найдена ни в самом классе, ни в его базовых классах, вызов откомпилирован не будет.
- Как только имя `mem` будет найдено, осуществляется обычная проверка соответствия типов (см. раздел 6.1, стр. 269), гарантирующая допустимость найденного определения для данного вызова.
- Если вызов допустим, компилятор создает код, зависящий от того, является ли вызываемая функция виртуальной или нет:
 - Если функция `mem()` виртуальная и вызов осуществляется через ссылку или указатель, то компилятор создает код, который во время выполнения определяет на основании динамического типа объекта выполняемую версию функции.
 - В противном случае, если функция не является виртуальной или если вызов осуществляется для объекта (а не ссылки или указателя), то компилятор создает код обычного вызова функции.

Как обычно, поиск имени осуществляется перед проверкой соответствия типов

Как уже упоминалось, функции, объявленные во внутренней области видимости, не перегружают функции, объявленные во внешней области видимости (см. раздел 6.4.1, стр. 307). В результате функции, определенные в про-

изводном классе, *не* перегружают функции-члены, определенные в его базовом классе (классах). Подобно любой другой области видимости, если имя члена производного класса (т.е. определенное во внутренней области видимости) совпадает с именем члена базового класса (т.е. именем во внешней области видимости), то в рамках производного класса имя, определенное в производном классе, скрывает имя в базовом классе. Имя функции-члена базового класса скрывается, даже если у функций будут разные списки параметров:

```
struct Base {
    int memfcn();
};
struct Derived : Base {
    int memfcn(int); // скрывает memfcn() в базовом классе
};
Derived d; Base b;
b.memfcn();          // вызов Base::memfcn()
d.memfcn(10);         // вызов Derived::memfcn()
d.memfcn();           // ошибка: memfcn() без аргументов скрывается
d.Base::memfcn();     // ok: вызов Base::memfcn()
```

Объявление функции `memfcn()` в классе `Derived` скрывает объявление функции `memfcn()` в классе `Base`. Не удивительно, что первый вызов через объект `b` класса `Base` вызывает версию в базовом классе. Точно так же второй вызов (через объект `d`) вызывает версию класса `Derived`. Удивительно то, что третий вызов, `d.memfcn()`, некорректен.

Чтобы распознать этот вызов, компилятор ищет имя `memfcn` в классе `Derived`. Этот класс определяет член по имени `memfcn`, и поиск на этом останавливается. Как только имя будет найдено, компилятор далее не ищет. Версия функции `memfcn()` в классе `Derived` ожидает аргумент типа `int`. Поскольку данный вызов такого аргумента не предоставляет, вызов ошибочен.



Виртуальные функции и область видимости

Теперь можно разобраться, почему у виртуальных функций должен быть одинаковый список параметров в базовом и производном классах (см. раздел 15.3, стр. 765). Если функции-члены в базовом и производном классах будут получать разные аргументы, не будет никакого способа вызвать версию производного класса через ссылку или указатель на базовый. Например:

```
class Base {
public:
    virtual int fcn();
};
class D1 : public Base {
public:
    // скрывает fcn() в базовом; функция fcn() не виртуальна
    // D1 наследует определение из Base::fcn()
    int fcn(int);          // список параметров fcn() в Base другой
    virtual void f2();     // новая виртуальная функция, не существующая Base
```



```
};
class D2 : public D1 {
public:
    int fcn(int); // неvirtуальная функция скрывает D1::fcn(int)
    int fcn();    // переопределяет виртуальную функцию fcn() из Base
    void f2();    // переопределяет виртуальную функцию f2() из D1
};
```

Функция `fcn()` в классе `D1` не переопределяет виртуальную функцию `fcn()` из класса `Base`, поскольку у них разные списки параметров. Вместо этого она *скрывает* функцию `fcn()` из базового класса. Фактически у класса `D1` есть две функции по имени `fcn()`: класс `D1` унаследовал виртуальную функцию `fcn()` от класса `Base`, а также определяет собственную неvirtуальную функцию-член по имени `fcn()`, получающую параметр типа `int`.

Вызов скрытой виртуальной функции через базовый класс

С учетом классов, описанных выше, рассмотрим несколько разных способов вызова этих функций:

```
Base bobj; D1 d1obj; D2 d2obj;
Base *bp1 = &bobj, *bp2 = &d1obj, *bp3 = &d2obj;
bp1->fcn(); // виртуальный вызов Base::fcn() во время выполнения
bp2->fcn(); // виртуальный вызов Base::fcn() во время выполнения
bp3->fcn(); // виртуальный вызов D2::fcn() во время выполнения
D1 *d1p = &d1obj; D2 *d2p = &d2obj;
bp2->f2(); // ошибка: Base не имеет члена по имени f2()
d1p->f2(); // виртуальный вызов D1::f2() во время выполнения
d2p->f2(); // виртуальный вызов D2::f2() во время выполнения
```

Все три первых вызова сделаны через указатели на базовый класс. Поскольку функция `fcn()` является виртуальной, компилятор создает код, способный во время выполнения решить, какую версию вызвать.

Это решение будет принято на основании фактического типа объекта, с которым связан указатель. В случае указателя `bp2` основной объект имеет тип `D1`. Этот класс не переопределяет функцию `fcn()` без параметров. Таким образом, вызов через указатель `bp2` распознается (во время выполнения) как версия, определенная в классе `Base`.

Следующие три вызова осуществляются через указатели с отличными типами. Каждый указатель указывает на один из типов в этой иерархии. Первый вызов некорректен, так как в классе `Base` нет функции `f2()`. Тот факт, что указатель случайно указывает на производный объект, является несущественным.

И наконец, рассмотрим вызовы неvirtуальной функции `fcn(int)`:

```
Base *p1 = &d2obj; D1 *p2 = &d2obj; D2 *p3 = &d2obj;
p1->fcn(42); // ошибка: Base не имеет версии fcn(), получающей int
p2->fcn(42); // статическое связывание, вызов D1::fcn(int)
p3->fcn(42); // статическое связывание, вызов D2::fcn(int)
```

В каждом вызове указатель случайно указывает на объект типа D2. Но динамический тип не имеет значения, когда происходит вызов неvirtуальной функции. Вызываемая версия зависит только от статического типа указателя.

Переопределение перегруженных функций

Подобно любой другой функции, функция-член (виртуальная или нет) может быть перегружена. Производный класс способен переопределить любое количество экземпляров перегруженных функций, которые он унаследовал. Если производный класс желает сделать все перегруженные версии доступными через свой тип, то он должен переопределить их все или ни одну из них.

Иногда класс должен переопределить некоторые, но не все функции в наборе перегруженных. В таких случаях было бы весьма утомительно переопределять каждую версию базового класса, чтобы переопределить только те, которые должен специализировать класс.

Вместо переопределения каждой версии базового класса, которую он унаследовал, производный класс может предоставить объявление `using` (см. раздел 15.5, стр. 776) для перегруженного члена. Объявление `using` определяет только имя; оно не может определить список параметров. Таким образом, объявление `using` для функции-члена базового класса добавляет все перегруженные экземпляры этой функции в область видимости производного класса. Перенеся все имена в свою область видимости, производный класс должен определить только те функции, которые действительно зависят от его типа.

Обычные правила объявления `using` в классе относятся и к именам перегруженных функций (см. раздел 15.5, стр. 776); каждый перегруженный экземпляр функции в базовом классе должен быть доступен в производном классе. Доступ к перегруженным версиям, которые в противном случае не переопределяются производным классом, будет возможен в точке объявления `using`.

Упражнения раздела 15.6

Упражнение 15.23. Предположим, что определенный на стр. 782 класс D1 намеревается переопределить свою унаследованную функцию `fcn()`. Как исправить этот класс? Предположим, что класс исправлен так, что функция `fcn()` соответствует определению в классе `Base`. Как бы распознавались вызовы в этом разделе?

15.7. Конструкторы и функции управления копированием

Подобно любому другому классу, класс в иерархии наследования контролирует происходящее при создании, копировании, перемещении, присвоении или

удалении объектов его типа. Как и у любого другого класса, если класс (базовый или производный) сам не определяет одну из функций управления копированием, ее синтезирует компилятор. Кроме того, как обычно, синтезируемая версия любой из этих функций-членов может быть удаленной функцией.



15.7.1. Виртуальные деструкторы

Основное воздействие, которое наследование оказывает на управление копированием для базового класса, заключается в том, что базовый класс обычно должен определять виртуальный деструктор (см. раздел 15.2.1, стр. 750). Деструктор должен быть виртуальной функцией, чтобы обеспечить объектам в иерархии наследования возможность динамического создания.

Помните, что деструктор выполняется при удалении указателя на динамически созданный объект (см. раздел 13.1.3, стр. 637). Если это указатель на тип в иерархии наследования, вполне возможно, что статический тип указателя может отличаться от динамического типа удаляемого объекта (см. раздел 15.2.2, стр. 754). Например, при удалении указателя типа `Quote*` может оказаться, что он указывал на объект класса `Bulk_quote`. Если он указывает на объект типа `Bulk_quote`, компилятор должен знать, что следует выполнить деструктор именно класса `Bulk_quote`. Подобно любой другой функции, чтобы был выполнен надлежащий деструктор, в базовом классе его следует определить как виртуальную функцию:

```
class Quote {
public:
    // виртуальный деструктор необходим при удалении указателя на
    // базовый тип, указывающего на объект производного
    virtual ~Quote() = default; // динамическое связывание для
                                // деструктора
};
```

Подобно любой другой виртуальной функции, виртуальный характер деструктора наследуется. Таким образом, у классов, производных от класса `Quote`, окажутся виртуальные деструкторы, будь то синтезируемый деструктор или собственный. Пока деструктор базового класса остается виртуальной функцией, при удалении указателя на базовый класс будет выполнен соответствующий деструктор:

```
Quote *itemP = new Quote; // статический и динамический типы совпадают
delete itemP;             // вызов деструктора для Quote
itemP = new Bulk_quote;   // статический и динамический типы разные
delete itemP;             // вызов деструктора для Bulk_quote
```



Выполнение оператора `delete` для указателя на базовый класс, который указывает на объект производного класса, приведет к непредсказуемым последствиям, если деструктор базового класса не будет виртуальным.

Деструкторы базовых классов — важное исключение из эмпирических правил, согласно которым, если класс нуждается в деструкторе, то он также нуждается в функциях копирования и присвоения (см. раздел 13.6, стр. 640). Базовый класс почти всегда нуждается в деструкторе, поэтому он может сделать деструктор виртуальным. Если базовый класс обладает пустым деструктором, только чтобы сделать его виртуальным, то наличие у класса деструктора вовсе не означает, что также необходим оператор присвоения или конструктор копий.

Виртуальный деструктор отменяет синтез функций перемещения

Тот факт, что базовый класс нуждается в виртуальном деструкторе, имеет важное косвенное последствие для определения базовых и производных классов: если класс определит деструктор (даже с использованием синтаксиса `= default`, чтобы использовать синтезируемую версию), то компилятор не будет синтезировать функцию перемещения для этого класса (см. раздел 13.6.2, стр. 680).

Упражнения раздела 15.7.1

Упражнение 15.24. Какие виды классов нуждаются в виртуальном деструкторе? Какие задачи должен выполнять виртуальный деструктор?



15.7.2. Синтезируемые функции управления копированием и наследование

Синтезируемые функции-члены управления копированием в базовом или производном классе выполняются, как любой другой синтезируемый конструктор, оператор присвоения или деструктор: они почленно инициализируют, присваивают или удаляют члены самого класса. Кроме того, эти синтезируемые члены инициализируют, присваивают или удаляют прямую базовую часть объекта при помощи соответствующей функции базового класса. Соответствующие примеры приведены ниже.

- Синтезируемый стандартный конструктор класса `Bulk_quote` запускает стандартный конструктор класса `Disc_Quote`, который в свою очередь запускает стандартный конструктор класса `Quote`.
- Стандартный конструктор класса `Quote` инициализирует по умолчанию переменную-член `bookNo` пустой строкой и использует внутриклассовый инициализатор для инициализации переменной-члена `price` нулем.
- Когда конструктор класса `Quote` завершает работу, конструктор класса `Disc_Quote` продолжает ее, используя внутриклассовые инициализаторы для инициализации переменных `qty` и `discount`.

- Когда завершает работу конструктор класса `Disc_quote`, конструктор класса `Bulk_quote` продолжает ее, но не выполняет никаких других действий.

Точно так же синтезируемый конструктор копий класса `Bulk_quote` использует (синтезируемый) конструктор копий класса `Disc_quote`, который использует (синтезируемый) конструктор копий класса `Quote`. Конструктор копий класса `Quote` копирует переменные-члены `bookNo` и `price`; а конструктор копий класса `Disc_Quote` копирует переменные-члены `qty` и `discount`.

Следует заметить, что не имеет значения, синтезируется ли функция-член базового класса (как в случае иерархии `Quote`) или имеет предоставленное пользователем определение. Важно лишь то, что соответствующая функция-член доступна (см. раздел 15.5, стр. 772) и что она не удаленная.

Каждый из классов иерархии `Quote` использует синтезируемый деструктор. Производные классы делают это неявно, тогда как класс `Quote` делает это явно, определяя свой (виртуальный) деструктор как `= default`. Синтезируемый деструктор (как обычно) пуст, и его неявная часть удаляет члены класса (см. раздел 13.1.3, стр. 636). В дополнение к удалению собственных членов фаза удаления деструктора в производном классе удаляет также свою прямую базовую часть. Этот деструктор в свою очередь вызывает деструктор своего прямого базового класса, если он есть. И так далее до корневого класса иерархии.

Как уже упоминалось, у класса `Quote` нет синтезируемых функций перемещения, поскольку он определяет деструктор. При каждом перемещении объекта `Quote` (см. раздел 13.6.2, стр. 684) будут использоваться (синтезируемые) функции копирования. Как будет продемонстрировано ниже, тот факт, что у класса `Quote` нет функций перемещения, означает, что его производные классы также не будут их иметь.



Базовые классы и удаленные функции управления копированием в производном классе

Синтезируемый стандартный конструктор или любая из функций-членов управления копированием базового либо производного класса может быть определена как удаленная по тем же причинам, что и в любом другом классе (см. раздел 13.1.6, стр. 644 и раздел 13.6.2, стр. 680). Кроме того, способ определения базового класса может вынудить член производного класса стать удаленным.

- Если стандартный конструктор, конструктор копий, оператор присвоения копии или деструктор в базовом классе удалены или недоступны (раздел 15.5, стр. 773), то соответствующая функция-член в производном классе определяется как удаленная, поскольку компилятор не может использовать функцию-член базового класса для создания, присвоения или удаления части объекта базового класса.

- Если у базового класса недоступен или удален деструктор, то синтезируемые стандартный конструктор и конструктор копий в производных классах определяются как удаленные, поскольку нет никакого способа удалить базовую часть производного объекта.
- Как обычно, компилятор не будет синтезировать удаленную функцию перемещения. Если использовать синтаксис `= default` для создания функции перемещения, то это будет удаленная функция в производном классе, если соответствующая функция в базовом классе будет удалена или недоступна, поскольку часть базового класса не может быть перемещена. Конструктор перемещения также будет удален, если деструктор базового класса окажется удален или недоступен.

Для примера рассмотрим базовый класс В:

```
class B {
public:
    B();
    B(const B&) = delete;
    // другие члены, исключая конструктор перемещения
};
class D : public B {
    // нет конструкторов
};
D d;           // ok: синтезируемый стандартный конструктор класса D использует
               // стандартный конструктор класса B
D d2(d); // ошибка: синтезируемый конструктор копий класса D удален
D d3(std::move(d)); // ошибка: неявно использованный удаленный
                   // конструктор копий класса D
```

Класс имеет доступный стандартный конструктор и явно удаленный конструктор копий. Поскольку конструктор копий определяется, компилятор не будет синтезировать для класса В конструктор перемещения (см. раздел 13.6.2, стр. 680). В результате невозможно ни переместить, ни скопировать объекты типа В. Если бы класс, производный от типа В, хотел позволить своим объектам копирование или перемещение, то этот производный класс должен был бы определить свои собственные версии этих конструкторов. Конечно, этот класс должен был бы решить, как скопировать или переместить члены в эту часть базового класса. Практически, если у базового класса нет стандартного конструктора копий или конструктора перемещения, то его производные классы также обычно не будут их иметь.

Функции перемещения и наследование

Как уже упоминалось, большинство базовых классов определяет виртуальный деструктор. В результате по умолчанию базовые классы вообще не получают синтезируемых функций перемещения. Кроме того, по умолчанию классы, производные от базового класса, у которого нет функций перемещения, также не получают синтезируемых функций перемещения.

Поскольку отсутствие функции перемещения в базовом классе подавляет синтез функций перемещения в его производных классах, базовые классы обычно должны определять функции перемещения, если это имеет смысл. Наш класс `Quote` может использовать синтезируемые версии. Однако класс `Quote` должен определить эти члены явно. Как только он определит собственные функции перемещения, он должен будет также явно определить версии копирования (см. раздел 13.6.2, стр. 683):

```
class Quote {
public:
    Quote() = default;           // почленная инициализация по умолчанию
    Quote(const Quote&) = default; // почленное копирование
    Quote(Quote&&) = default;     // почленное копирование
    Quote& operator=(const Quote&) = default; // присвоение копии
    Quote& operator=(Quote&&) = default;     // перемещение
    virtual ~Quote() = default;
    // другие члены, как прежде
};
```

Теперь объекты класса `Quote` будут почленно копироваться, перемещаться, присваиваться и удаляться. Кроме того, классы, производные от класса `Quote`, также автоматически получают синтезируемые функции перемещения, если у них не будет членов, которые воспрепятствуют перемещению.

Упражнения раздела 15.7.2

Упражнение 15.25. Зачем определять стандартный конструктор для класса `Disc_quote`? Как повлияет на поведение класса `Bulk_quote`, если вообще повлияет, удаление этого конструктора?



15.7.3. Функции-члены управления копированием производного класса

Как упоминалось в разделе 15.2.2 (стр. 755), фаза инициализации конструктора производного класса инициализирует часть (части) базового класса производного объекта наряду с инициализацией его собственных членов. В результате конструкторы копирования и перемещения для производного класса должны копировать и перемещать члены своей базовой части наравне с производной. Точно так же оператор присвоения производного класса должен присваивать члены базовой части производного объекта.

В отличие от конструкторов и операторов присвоения, деструктор несет ответственность только за освобождение ресурсов, зарезервированных производным классом. Помните, что члены объекта освобождаются неявно (см. раздел 13.1.3, стр. 637). Точно так же часть базового класса объекта производного класса освобождается автоматически.



Когда производный класс определяет функцию копирования или перемещения, эта функция несет ответственность за копирование или перемещение всего объекта, включая члены базового класса.



Определение конструктора копии или перемещения производного класса

При определении конструктора копии или перемещения (см. раздел 13.1.1, стр. 630 и раздел 13.6.2, стр. 676) для производного класса обычно используется соответствующий конструктор базового класса, инициализирующий базовую часть объекта:

```
class Base { /* ... */ };
class D: public Base {
public:
    // по умолчанию стандартный конструктор базового класса
    // инициализирует базовую часть объекта
    // чтобы использовать конструктор копии или перемещения, его следует
    // вызвать явно
    // конструктор в списке инициализации конструктора
    D(const D& d): Base(d) // копирование базовых членов
        /* инициализаторы для членов класса D */ { /* ... */ }
    D(D&& d): Base(std::move(d)) // перемещение базовых членов
        /* инициализаторы для членов класса D */ { /* ... */ }
};
```

Инициализатор `Base(d)` передает объект класса `D` конструктору базового класса. Хотя в принципе у класса `Base` может быть конструктор с параметром типа `D`, на практике это очень маловероятно. Вместо этого инициализатор `Base(d)` будет (обычно) соответствовать конструктору копий класса `Base`. В этом конструкторе объект `d` будет связан с параметром типа `Base&`. Конструктор копий класса `Base` скопирует базовую часть объекта `d` в создаваемый объект. Будь инициализатор для базового класса пропущен, для инициализации базовой части объекта класса `D` будет использован стандартный конструктор класса `Base`.

```
// вероятно, неправильное определение конструктора копий D
// часть базового класса инициализируется по умолчанию, а не копией
D(const D& d) /* инициализаторы членов класса, но не базового класса */
{ /* ... */ }
```

Предположим, что конструктор класса `D` копирует производные члены объекта `d`. Этот вновь созданный объект был бы настроен странно: его члены класса `Base` содержали бы значения по умолчанию, в то время как его члены класса `D` были бы копиями данных из другого объекта.



По умолчанию стандартный конструктор базового класса инициализирует часть базового класса объекта производного. Если необходимо копирование (или перемещение) части базового класса, следует явно использовать конструктор копий (или перемещения) для базового класса в списке инициализации конструктора производного.

Оператор присвоения производного класса

Подобно конструктору копирования и перемещения, оператор присвоения производного класса (см. раздел 13.1.2, стр. 634 и раздел 13.6.2, стр. 679) должен присваивать свою базовую часть явно:

```
// Base::operator=(const Base&) не вызывается автоматически
D &D::operator=(const D &rhs)
{
    Base::operator=(rhs); // присваивает базовую часть
    // присвоение членов в производном классе, как обычно,
    // отработка самоприсвоения и освобождения ресурсов
    return *this;
}
```

Этот оператор начинается с явного вызова оператора присвоения базового класса, чтобы присвоить члены базовой части объекта производного. Оператор базового класса (по-видимому, правильно) отработает случай присвоения себя себе и, если нужно, освободит прежнее значение в базовой части левого операнда и присвоит новое значение правой. По завершении работы оператора продолжается выполнение всего необходимого для присвоения членов в производном классе.

Следует заметить, что конструктор или оператор присвоения производного класса может использовать соответствующую функцию базового класса независимо от того, определил ли базовый класс собственную версию этого оператора или использует синтезируемую. Например, вызов оператора `Base::operator=` выполняет оператор присвоения копии в классе `Base`. При этом несущественно, определяется ли этот оператор классом `Base` явно или синтезируется компилятором.

Деструктор производного класса

Помните, переменные-члены объекта неявно удаляются после завершения выполнения тела деструктора (см. раздел 13.1.3, стр. 637). Точно так же части базового класса объекта тоже удаляются неявно. В результате, в отличие от конструкторов и операторов присвоения, производный деструктор отвечает за освобождение только тех ресурсов, которые зарезервировал производный класс:

```
class D: public Base {  
public:  
    // Base::~Base вызывается автоматически  
    ~D() { /* освободить члены производного класса */ }  
};
```

Объекты удаляются в порядке, противоположном их созданию: сначала выполняется деструктор производного класса, а затем деструкторы базового класса, назад по иерархии наследования.

Вызовы виртуальных функций в конструкторах и деструкторах

Как уже упоминалось, сначала создается часть базового класса в объекте производного. Пока выполняется конструктор базового класса, производная часть объекта остается неинициализированной. Точно так же производные объекты удаляются в обратном порядке, чтобы при выполнении деструктора базового класса производная часть уже была удалена. В результате на момент выполнения членов базового класса объект оказывается в незавершенном состоянии.

Чтобы приспособиться к этой незавершенности, компилятор рассматривает объект как изменяющий свой тип во время создания или удаления. Таким образом, во время создания объекта он считается объектом того же класса, что и конструктор; вызовы виртуальной функции будут связаны так, как будто у объекта тот же тип, что и у самого конструктора. Аналогично для деструктора. Эта привязка относится к виртуальным функциям, вызванным непосредственно или косвенно, из функции, которую вызывает конструктор (или деструктор).

Чтобы понять это поведение, рассмотрим, что произошло бы, если бы версия виртуальной функции производного класса была вызвана из конструктора базового класса. Эта виртуальная функция, вероятно, обратится к членам производного объекта. В конце концов, если бы виртуальная функция не должна была использовать члены производного объекта, то производный класс, вероятно, мог бы использовать ее версию в базовом классе. Но во время выполнения конструктора базового класса эти члены остаются неинициализированными. Если бы такой доступ был разрешен, то работа программы, вероятно, закончилась бы катастрофически.



Если конструктор или деструктор вызывает виртуальную функцию, то выполняемая версия будет соответствовать типу самого конструктора или деструктора.

Упражнения раздела 15.7.3

Упражнение 15.26. Определите для классов `Quote` и `Bulk_quote` функции-члены управления копированием, осуществляющие те же действия, что и синтезируемые версии. Снабдите их и другие конструкторы операторами вывода, идентифицирующими выполняемую функцию. Напишите программу с использованием этих классов и укажите, какие объекты будут созданы и удалены. Сравните свои предположения с выводом и продолжите экспериментировать, пока ваши предположения не станут правильными.



15.7.4. Унаследованные конструкторы

По новому стандарту производный класс может многократно использовать конструкторы, определенные его прямым базовым классом. Хотя, как будет продемонстрировано далее, такие конструкторы не наследуются в обычном смысле этого слова, о них, тем не менее, говорят как об унаследованных. По тем же причинам, по которым класс может инициализировать только свой прямой базовый класс, класс может наследовать конструкторы только от своего прямого базового класса. Класс не может унаследовать стандартный конструктор, конструктор копий и перемещения. Если производный класс не определяет эти конструкторы сам, то компилятор синтезирует их, как обычно.

Производный класс наследует конструкторы своего базового класса при помощи объявления `using`, в котором указан его (прямой) базовый класс. В качестве примера можно переопределить класс `Bulk_quote` (см. раздел 15.4, стр. 771) так, чтобы он унаследовал конструкторы от класса `Disc_quote`:

```
class Bulk_quote : public Disc_quote {
public:
    using Disc_quote::Disc_quote; // наследует конструкторы Disc_quote
    double net_price(std::size_t) const;
};
```

Обычно объявление `using` просто делает имя видимым в текущей области видимости. Применительно к конструктору объявление `using` приводит к созданию компилятором кода. Компилятор создает в производном классе конструктор, соответствующий каждому конструктору в базовом классе. Таким образом, для каждого конструктора в базовом классе компилятор создает в производном классе конструктор с таким же списком параметров.

Эти созданные компилятором конструкторы имеют такую форму:

```
производный(параметры) : базовый(аргументы) { }
```

где *производный* — имя производного класса; *базовый* — имя базового класса; *параметры* — список параметров конструктора; *аргументы* передают параметры из конструктора производного класса в конструктор базового. В классе `Bulk_quote` унаследованный конструктор был бы эквивалентен следующему:

```
Bulk_quote(const std::string& book, double price,  
           std::size_t qty, double disc):  
    Disc_quote(book, price, qty, disc) { }
```

Если у производного класса есть какие-нибудь собственные переменные-члены, они инициализируются по умолчанию (см. раздел 7.1.4, стр. 346).

Характеристики унаследованного конструктора

В отличие от объявлений `using` для обычных членов, объявление `using` для конструктора не изменяет уровень доступа унаследованного конструктора (конструкторов). Например, независимо от того, где расположено объявление `using`, закрытый конструктор в базовом классе остается закрытым в производном; то же относится к защищенным и открытым конструкторам.

Кроме того, объявление `using` не может использовать определение как `explicit` или `constexpr`. Если конструктор объявлен как `explicit` (см. раздел 7.5.4, стр. 383) или `constexpr` (см. раздел 7.5.6, стр. 385) в базовом классе, у унаследованного конструктора будет то же свойство.

Если у конструктора базового класса есть аргументы по умолчанию (см. раздел 6.5.1, стр. 308), они не наследуются. Вместо этого производный класс получает несколько унаследованных конструкторов, в которых каждый параметр с аргументом по умолчанию благополучно пропущен. Например, если у базового класса будет конструктор с двумя параметрами, у второго из которых будет аргумент по умолчанию, то производный класс получит два конструктора: один с обоими параметрами (и никакого аргумента по умолчанию) и второй конструктор с одним параметром, соответствующим левому параметру без аргумента по умолчанию в базовом классе.

Если у базового класса есть несколько конструкторов, то за двумя исключениями производный класс унаследует каждый из конструкторов своего базового класса. Первое исключение — производный класс может унаследовать некоторые конструкторы и определить собственные версии других конструкторов. Если производный класс определяет конструктор с теми же параметрами, что и конструктор в базовом классе, то этот конструктор не наследуется. Конструктор, определенный в производном классе, используется вместо унаследованного конструктора.

Второе исключение — стандартный конструктор, конструктор копий и конструктор перемещения не наследуются. Эти конструкторы синтезируются с использованием обычных правил. Унаследованный конструктор не рассматривается как пользовательский конструктор. Поэтому у класса, который содержит только унаследованные конструкторы, будет синтезируемый стандартный конструктор.

Упражнения раздела 15.7.4

Упражнение 15.27. Переопределите свой класс `Bulk_quote` так, чтобы унаследовать его конструкторы.



15.8. Контейнеры и наследование

При использовании контейнера для хранения объектов из иерархии наследования их обычно хранят косвенно. Нельзя поместить объекты связанных наследованием типов непосредственно в контейнер, поскольку нет никакого способа определить контейнер, содержащий элементы разных типов.

В качестве примера определим вектор, содержащий несколько объектов для книг, которые клиент желает купить. Вполне очевидно, что не получится использовать вектор, содержащий объекты класса `Bulk_quote`. Нельзя преобразовать объекты класса `Quote` в объекты класса `Bulk_quote` (см. раздел 15.2.3, стр. 760), поэтому объекты класса `Quote` в этот вектор поместить не получится.

Может быть и не так очевидно, но вектор объектов типа `Quote` также нельзя использовать. В данном случае можно поместить объекты класса `Bulk_quote` в контейнер, но эти объекты перестанут быть объектами класса `Bulk_quote`:

```
vector<Quote> basket;  
basket.push_back(Quote("0-201-82470-1", 50));  
// ok, но в basket копируется только часть Quote объекта  
basket.push_back(Bulk_quote("0-201-54848-8", 50, 10, .25));  
// вызовов версии, определенной в Quote, выводит 750, т.е. 15 * $50  
cout << basket.back().net_price(15) << endl;
```

Элементами вектора `basket` являются объекты класса `Quote`. Когда в вектор добавляется объект класса `Bulk_quote`, его производная часть игнорируется (см. раздел 15.2.3, стр. 762).



ВНИМАНИЕ

Поскольку при присвоении объекту базового класса объект производного класса усекается, контейнеры не очень удобны для хранения объектов разных классов, связанных наследственными отношениями.

Помещайте в контейнеры указатели (интеллектуальные), а не объекты

Когда необходим контейнер, содержащий объекты, связанные наследованием, как правило, определяют контейнер указателей (предпочтительно интеллектуальных (см. раздел 12.1, стр. 574)) на базовый класс. Как обычно, динамический тип объекта, на который указывает этот указатель, мог бы быть типом базового класса или типом, производным от него:

```
vector<shared_ptr<Quote>> basket;
basket.push_back(make_shared<Quote>("0-201-82470-1", 50));
basket.push_back(
    make_shared<Bulk_quote>("0-201-54848-8", 50, 10, .25));
// вызовов версии, определенной в Quote, выводит 562.5,
// т.е. со скидкой, меньше, чем 15 * $50
cout << basket.back()->net_price(15) << endl;
```

Поскольку вектор `basket` содержит указатели `shared_ptr`, для получения объекта, функция `net_price()` которого выполнится, следует обратиться к значению, возвращенному функцией `basket.back()`. Для этого в вызове функции `net_price()` используется оператор `->`. Как обычно, вызываемая версия функции `net_price()` зависит от динамического типа объекта, на который указывает этот указатель.

Следует заметить, что вектор `basket` был определен как `shared_ptr<Quote>`, все же во втором вызове функции `push_back()` был передан указатель на объект класса `Bulk_quote`. Подобно тому, как можно преобразовать обычный указатель на производный тип в указатель на тип базового класса (см. раздел 15.2.2, стр. 754), можно также преобразовать интеллектуальный указатель на производный тип в интеллектуальный указатель на тип базового класса. Таким образом, вызов функции `make_shared<Bulk_quote>()` возвращает объект `shared_ptr<Bulk_quote>`, в который преобразуется `shared_ptr<Quote>` при вызове функции `push_back()`. В результате, несмотря на внешний вид, у всех элементов вектора `basket` будет тот же тип.

Упражнения раздела 15.8

Упражнение 15.28. Определите вектор для содержания объектов класса `Quote`, но поместите в него объекты класса `Bulk_quote`. Вычислите общую сумму результатов вызова функции `net_price()` для всех элементов вектора.

Упражнение 15.29. Повторите предыдущую программу, но на сей раз храните указатели `shared_ptr` на объекты типа `Quote`. Объясните различие в сумме данной версии программы и предыдущей. Если никакой разницы нет, объясните почему.



15.8.1. Разработка класса `Basket`

Ирония объектно-ориентированного программирования на языке C++ в том, что невозможно использовать объекты непосредственно. Вместо них приходится использовать указатели и ссылки. Поскольку указатели усложняют программы, зачастую приходится определять вспомогательные классы, чтобы избежать осложнений. Для начала определим класс, представляющий корзину покупателя:

```

class Basket {
public:
    // Basket использует синтезируемый стандартный конструктор и
    // функции-члены управления копированием
    void add_item(const std::shared_ptr<Quote> &sale)
        { items.insert(sale); }
    // выводит общую стоимость каждой книги и общий счет для всех
    // товаров в корзине
    double total_receipt(std::ostream&) const;
private:
    // функция сравнения shared_ptrs, необходимая элементам
    // набора multiset
    static bool compare(const std::shared_ptr<Quote> &lhs,
                        const std::shared_ptr<Quote> &rhs)
        { return lhs->isbn() < rhs->isbn(); }
    // набор multiset содержит несколько стратегий расценок,
    // упорядоченных по сравниваемому элементу
    std::multiset<std::shared_ptr<Quote>, decltype(compare)*>
        items{compare};
}

```

Для хранения транзакций класс использует контейнер `multiset` (см. раздел 11.2.1, стр. 541), позволяющий содержать несколько транзакций по той же книге, чтобы все транзакции для данной книги находились вместе (см. раздел 11.2.2, стр. 543).

Элементами контейнера `multiset` будут указатели `shared_ptr`, и для них нет оператора “меньше”. В результате придется предоставить собственный оператор сравнения для упорядочивания элементов (см. раздел 11.2.2, стр. 543). Здесь определяется закрытая статическая функция-член `compare()`, сравнивающая `isbn` объектов, на которые указывают указатели `shared_ptr`. Инициализируем контейнер `multiset` с использованием этой функции сравнения и внутриклассового инициализатора (см. раздел 7.3.1, стр. 357):

```

// набор multiset содержит несколько стратегий расценок,
// упорядоченных по сравниваемому элементу
std::multiset<std::shared_ptr<Quote>, decltype(compare)*>
    items{compare};

```

Это объявление может быть трудно понять, но, читая его слева направо, можно заметить, что определяется контейнер `multiset` указателей `shared_ptr` на объекты класса `Quote`. Для упорядочивания элементов контейнер `multiset` будет использовать функцию с тем же типом, что и функция-член `compare()`. Элементами контейнера `multiset` будут объекты `items`, которые инициализируются для использования функции `compare()`.

Определение членов класса `Basket`

Класс `Basket` определяет только две функции. Функция-член `add_item()` определена в классе. Она получает указатель `shared_ptr` на динамически

созданный объект класса `Quote` и помещает его в контейнер `multiset`. Вторая функция-член, `total_receipt()`, выводит полученный счет для содержимого корзины и возвращает цену за все элементы в ней:

```
double Basket::total_receipt(ostream &os) const
{
    double sum = 0.0; // содержит текущую сумму

    // iter ссылается на первый элемент в пакете элементов с тем же ISBN
    // upper_bound() возвращает итератор на элемент сразу после
    // конца этого пакета
    for (auto iter = items.cbegin();
         iter != items.cend();
         iter = items.upper_bound(*iter)) {
        // известно, что в Basket есть по крайней мере один элемент
        // с этим ключом
        // вывести строку для элемента этой книги
        sum += print_total(os, **iter, items.count(*iter));
    }
    os << "Total Sale: " << sum << endl; // вывести в конце общий счет
    return sum;
}
```

Цикл `for` начинается с определения и инициализации итератора `iter` на первый элемент контейнера `multiset`. Условие проверяет, не равен ли `iter` значению `items.cend()`. Если да, то обработаны все покупки и цикл `for` завершается. В противном случае обрабатывается следующая книга.

Интересный момент — выражение “инкремента” в цикле `for`. Это не обычный цикл, читающий каждый элемент и перемещающий итератор `iter` на следующий. При вызове функции `upper_bound()` (см. раздел 11.3.5, стр. 560) он перескакивает через все элементы, которые соответствуют текущему ключу. Вызов функции `upper_bound()` возвращает итератор на элемент сразу после последнего с тем же ключом, что и `iter`. Возвращаемый итератор обозначает или конец набора, или следующую книгу.

Для вывода подробностей по каждой книге в корзине в цикле `for` происходит вызов функции `print_total()` (см. раздел 15.1, стр. 749):

```
sum += print_total(os, **iter, items.count(*iter));
```

Аргументами функции `print_total()` являются поток `ostream` для записи, обрабатываемый объект `Quote` и счет. При обращении к значению итератора `iter` возвращается указатель `shared_ptr`, указывающий на объект, который предстоит вывести. Чтобы получить этот объект, следует обратиться к значению этого указателя `shared_ptr`. Таким образом, выражение `**iter` возвращает объект класса `Quote` (или класса производного от него). Для выяснения количества элементов в контейнере `multiset` с тем же ключом (т.е. с тем же ISBN) используется его функция-член `count()` (см. раздел 11.3.5, стр. 558).

Как уже упоминалось, функция `print_total()` осуществляет вызов виртуальной функции `net_price()`, поэтому полученная цена зависит от динамического типа `**iter`. Функция `print_total()` выводит общую сумму для данной книги и возвращает вычисленную общую стоимость. Результат добавляется в переменную `sum`, которая выводится после завершения цикла `for`.

Соккрытие указателей

Пользователи класса `Basket` все еще должны иметь дело с динамической памятью, поскольку функция `add_item()` получает указатель `shared_ptr`. В результате пользователи вынуждены писать код так:

```
Basket bsk;
bsk.add_item(make_shared<Quote>("123", 45));
bsk.add_item(make_shared<Bulk_quote>("345", 45, 3, .15));
```

На следующем этапе переопределим функцию `add_item()` так, чтобы она получала объект класса `Quote` вместо указателя `shared_ptr`. Эта новая версия функции `add_item()` отработает резервирование памяти так, чтобы пользователи больше не должны были делать это сами. Определим две ее версии: одна будет копировать переданный ей объект, а другая перемещать его (см. раздел 13.6.3, стр. 690):

```
void add_item(const Quote& sale); // копирует переданный объект
void add_item(Quote&& sale);      // перемещает переданный объект
```

Единственная проблема в том, что функция `add_item()` не знает, какой тип резервировать. При резервировании памяти функция `add_item()` копирует (или переместит) свой параметр `sale`. Выражение `new` будет выглядеть примерно так:

```
new Quote(sale)
```

К сожалению, это выражение будет неправильным: оператор `new` резервирует объект запрошенного типа. Оно резервирует объект типа `Quote` и копирует часть `Quote` параметра `sale`. Но если переданный параметру `sale` объект будет иметь тип `Bulk_quote`, то он будет усечен.



Имитация виртуального копирования

Эту проблему можно решить, снабдив класс `Quote` виртуальной функцией-членом, резервирующей его копию.

```
class Quote {
public:
    // виртуальная функция, возвращающая динамически созданную копию
    // эти члены используют квалификаторы ссылки; раздел 13.6.3
    // (стр. 691)
    virtual Quote* clone() const & {return new Quote(*this);}
    virtual Quote* clone() &&
```

```

        {return new Quote(std::move(*this));}
    // другие члены как прежде
};

class Bulk_quote : public Quote {
    Bulk_quote* clone() const & {return new Bulk_quote(*this);}
    Bulk_quote* clone() &&
        {return new Bulk_quote(std::move(*this));}
    // другие члены, как прежде
};

```

Поскольку функция `add_item()` имеет версии копирования и перемещения, были определены версии l- и r-значения функции `clone()` (см. раздел 13.6.3, стр. 691). Каждая функция `clone()` резервирует новый объект ее собственного типа. Функция-член константной ссылки на l-значение копирует себя во вновь зарезервированный объект; функция-член ссылки на r-значение перемещает свои данные.

Используя функцию `clone()`, довольно просто написать новые версии функции `add_item()`:

```

class Basket {
public:
    void add_item(const Quote& sale) // копирует переданный объект
        { items.insert(std::shared_ptr<Quote>(sale.clone())); }
    void add_item(Quote&& sale)      // перемещает переданный объект
        { items.insert(
            std::shared_ptr<Quote>(std::move(sale).clone())); }
    // другие члены, как прежде
};

```

Как и сама функция `add_item()`, функция `clone()` перегружается на основании того, вызвана ли она для l- или r-значения. Таким образом, первая версия функции `add_item()` вызывает константную версию l-значения функции `clone()`, а вторая версия вызывает версию ссылки на r-значение. Обратите внимание, что хотя в версии r-значения типом параметра `sale` является ссылка на r-значение, сам параметр `sale` (как и любая другая переменная) является l-значением (см. раздел 13.6.1, стр. 675). Поэтому для привязки ссылки на r-значение к параметру `sale` вызывается функция `move()`.

Наша функция `clone()` является также виртуальной. Будет ли выполнена функция из класса `Quote` или `Bulk_quote`, зависит (как обычно) от динамического типа параметра `sale`. Независимо от того, копируются или перемещаются данные, функция `clone()` возвращает указатель на вновь зарезервированный объект его собственного типа. С этим объектом связывается указатель `shared_ptr`, и вызывается функция `insert()` для добавления этого вновь зарезервированного объекта к `items`. Обратите внимание: так как указатель `shared_ptr` поддерживает преобразование производного класса в базовый (см. раздел 15.2.2, стр. 754), указатель `shared_ptr<Quote>` можно привязать к `Bulk_quote*`.

Упражнения раздела 15.8.1

Упражнение 15.30. Напишите собственную версию класса `Basket` и используйте ее для вычисления цены за те же транзакции, что и в предыдущих упражнениях.

15.9. Возвращаясь к запросам текста

В качестве последнего примера наследования дополним приложение текстового запроса из раздела 12.3 (стр. 617). Написанные в этом разделе классы позволят искать вхождения данного слова в файле. Дополним эту систему возможностью создавать более сложные запросы. В этих примерах запросы будут выполняться к тексту следующей истории:

```
Alice Emma has long flowing red hair.
Her Daddy says when the wind blows
through her hair, it looks almost alive,
like a fiery bird in flight.
A beautiful fiery bird, he tells her,
magical but untamed.
"Daddy, shush, there is no such thing,"
she tells him, at the same time wanting
him to tell her more.
Shyly, she asks, "I mean, Daddy, is there?"
```

Система должна поддерживать следующие запросы.

- Запросы слов находят все строки, соответствующие заданной строке:
 Executing Query for: Daddy
 Daddy occurs 3 times
 (line 2) Her Daddy says when the wind blows
 (line 7) "Daddy, shush, there is no such thing,"
 (line 10) Shyly, she asks, "I mean, Daddy, is there?"
- Инверсный запрос с использованием оператора `~` возвращает строки, которые не содержат заданную строку:
 Executing Query for: ~(Alice)
 ~(Alice) occurs 9 times
 (line 2) Her Daddy says when the wind blows
 (line 3) through her hair, it looks almost alive,
 (line 4) like a fiery bird in flight.
 ...
- Запросы ИЛИ с использованием оператора `|` возвращают строки, содержащие любую из двух заданных строк:
 Executing Query for: (hair | Alice)
 (hair | Alice) occurs 2 times
 (line 1) Alice Emma has long flowing red hair.
 (line 3) through her hair, it looks almost alive,

- Запросы И с использованием оператора & возвращают строки, содержащие обе заданные строки:

```
Executing query for: (hair & Alice)
(hair & Alice) occurs 1 time
(line 1) Alice Emma has long flowing red hair.
```

Кроме того, нужна возможность объединить эти операторы так:

```
fiery & bird | wind
```

Для обработки составных выражений, таких как в этом примере, будут использованы обычные правила приоритета C++ (см. раздел 4.1.2, стр. 190). Таким образом, этому запросу соответствует строка, в которой присутствуют слова `fiery` и `bird` или слово `wind`:

```
Executing Query for: ((fiery & bird) | wind)
((fiery & bird) | wind) occurs 3 times
(line 2) Her Daddy says when the wind blows
(line 4) like a fiery bird in flight.
(line 5) A beautiful fiery bird, he tells her,
```

В отображаемом результате для указания способа интерпретации запроса используются круглые скобки. Подобно первоначальной реализации, система не должна отображать одинаковые строки несколько раз.

15.9.1. Объектно-ориентированное решение

Для представления запросов на поиск слов вполне логично было бы использовать класс `TextQuery` (см. раздел 12.3.2, стр. 620), а другие классы запросов можно было бы получить как производные от этого класса.

Однако такой подход неверен. Концептуально инверсный запрос не является разновидностью запроса на поиск слова. Инверсный запрос — это скорее запрос типа “имеет” (запрос на поиск слова или любой другой тип запроса), результат которого интерпретируется негативно.

Исходя из этого можно сделать вывод, что разные виды запросов следует оформить как независимые классы, которые совместно используют общий базовый класс:

```
WordQuery // Daddy
NotQuery  // ~Alice
OrQuery   // hair | Alice
AndQuery  // hair & Alice
```

Эти классы будут иметь только две функции.

- Функция `eval()`, получающая объект класса `TextQuery` и возвращающая объект класса `QueryResult`. Для поиска запрошенной строки функция `eval()` будет использовать переданный объект класса `TextQuery`.

- Функция `rep()`, возвращающая строковое представление базового запроса. Эту функцию использует функция `eval()` для создания объекта класса `QueryResult`, представляющего соответствия, а также оператор вывода, отображающий выражение запроса.

Абстрактный базовый класс

Как уже упоминалось, все четыре типа запроса не связаны друг с другом наследованием; концептуально они элементы одного уровня. Каждый класс использует тот же интерфейс, а значит, для представления этого интерфейса следует определить абстрактный базовый класс (см. раздел 15.4, стр. 771). Назовем этот абстрактный базовый класс `Query_base`, поскольку он должен служить корневым классом иерархии запроса.

КЛЮЧЕВАЯ КОНЦЕПЦИЯ. НАСЛЕДОВАНИЕ ИЛИ КОМПОЗИЦИЯ

Проектирование иерархии наследования — это достаточно сложная тема, которая выходит за рамки данного вводного курса. Однако имеет смысл упомянуть об одном достаточно важном факторе проектирования, с которым должен быть знаком каждый программист.

При определении класса как открыто производного от другого производный и базовый классы реализуют взаимоотношения типа “является” (*is a*). В хорошо проработанных иерархиях объекты открыто унаследованных классов применимы везде, где ожидается объект базового класса.

Еще одним популярным способом взаимоотношений классов является принцип “имеет” (*has a*). Типы, связанные отношениями “имеет”, подразумевают принадлежность.

В рассматриваемом примере с книжным магазином базовый класс представляет концепцию книги, продаваемой по предусмотренной цене, а класс `Bulk_quote` “является” конкретной книгой, продаваемой по розничной цене с определенной стратегией скидок. Классы приложения книжного магазина “имеют” цену и ISBN.

Класс `Query_base` определит функции `eval()` и `rep()` как чистые виртуальные (см. раздел 15.4, стр. 771). Каждый из классов, представляющих специфический вид запроса, должен переопределить эти функции. Классы `WordQuery` и `NotQuery` унаследуют непосредственно от класса `Query_base`. У классов `AndQuery` и `OrQuery` будет одна общая особенность, которой не будет у остальных классов в системе: у каждого будет по два операнда. Для моделирования этой особенности определим другой абстрактный базовый класс, `BinaryQuery`, представляющий запросы с двумя операндами. Классы `AndQuery` и `OrQuery` наследуются от класса `BinaryQuery`, который в свою очередь наследуется от класса `Query_base`. Результатом этих решений будет проект классов, представленный на рис. 15.2.

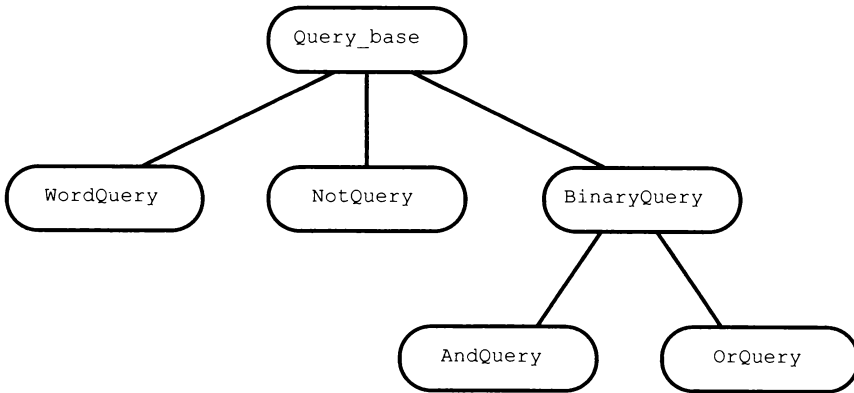


Рис. 15.2. Иерархия наследования `Query_base`

Соккрытие иерархии в классе интерфейса

Рассматриваемая программа будет обрабатывать запросы, а не создавать их. Но чтобы запустить программу на выполнение, необходимо определить способ создания запроса. Проще всего сделать это непосредственно в коде при помощи выражения C++. Например, чтобы создать описанный ранее составной запрос, можно использовать следующий код:

```
Query q = Query("fiery") & Query("bird") | Query("wind");
```

Это довольно сложное описание неявно предполагает, что код пользовательского уровня не будет использовать унаследованные классы непосредственно. Вместо этого будет создан класс интерфейса по имени `Query` (Запрос), который и скроет иерархию. Класс `Query` будет хранить указатель на класс `Query_base`. Этот указатель будет связан с объектом типа, производного от класса `Query_base`. Класс `Query` будет предоставлять те же функции, что и классы `Query_base`: функцию `eval()` для обработки соответствующего запроса и функцию `rep()` для создания строковой версии запроса. В нем также будет определен перегруженный оператор вывода, чтобы отображать соответствующий запрос.

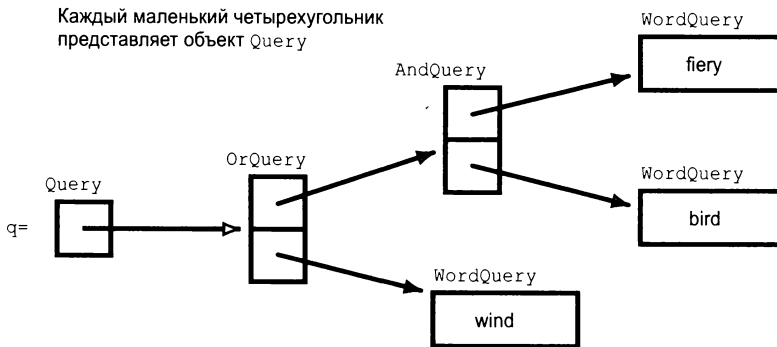
Пользователи будут создавать объекты класса `Query_base` и работать с ними только косвенно, через функции объектов класса `Query`. Для класса `Query`, наряду с получающим строку конструктором, определим три перегруженных оператора. Каждая из этих функций будет динамически резервировать новый объект типа, производного от класса `Query_base`:

- Оператор `&` создает объект класса `Query`, связанный с новым объектом класса `AndQuery`.
- Оператор `|` создает объект класса `Query`, связанный с новым объектом класса `OrQuery`.

- Оператор `~` создает объект класса `Query`, связанный с новым объектом класса `NotQuery`.
- Конструктор класса `Query`, получающий строку и создающий новый объект класса `WordQuery`.

Как работают эти классы

Следует понять, что работа этого приложения состоит в основном из построения объектов для представления запросов пользователя. Например, приведенное выше выражение создает коллекцию взаимодействовавших объектов, представленных на рис. 15.3.



Объекты создаются следующим выражением:
`Queryq=Query("fiery")&Query("bird")|Query("wind");`

Рис. 15.3. Объекты, созданные выражениями запросов

Как только создано дерево объектов, обработка (или отображение) данного запроса сводится к простому процессу (осуществляемому компилятором), который, следуя по линиям, опрашивает каждый объект дерева, чтобы выполнить (или отобразить) необходимые действия. Например, если происходит вызов функции `eval()` объекта `q` (т.е. корневого класса дерева), функция `eval()` опросит объект класса `OrQuery`, на который он указывает. Обработка этого объекта класса `OrQuery` приведет к вызову функции `eval()` для двух его операндов, что, в свою очередь, приведет к вызову функции `eval()` для объектов классов `AndQuery` и `WordQuery`, которые осуществляют поиск слова `wind`. Обработка объекта класса `AndQuery`, в свою очередь, приведет к обработке двух его объектов класса `WordQuery`, создав результаты для слов `fiery` и `bird` соответственно.

Новичкам в объектно-ориентированном программировании зачастую трудней всего разобраться в проекте программы. Но как только проект станет абсолютно понятен, его реализация не вызывает проблем. Чтобы проще было понять суть проекта, все используемые в этом примере классы были обобщены в табл. 15.1.

Таблица 15.1. Обзор проекта программы

Классы и операторы интерфейса программы запросов	
TextQuery	Класс, который читает указанный файл и создает карту поиска. Этот класс предоставляет функцию поиска <code>query()</code> , которая получает строковый аргумент и возвращает объект класса <code>QueryResult</code> , представляющий строки, в которых присутствует ее аргумент (см. раздел 12.3.2, стр. 620)
QueryResult	Класс, содержащий результаты вызова функции <code>query()</code> (см. раздел 12.3.2, стр. 622)
Query	Класс интерфейса, указывающий на объект типа, производного от класса <code>Query_base</code>
Query q(s)	Связывает объект <code>q</code> класса <code>Query</code> с новым объектом класса <code>WordQuery</code> , содержащим строку <code>s</code>
q1 & q2	Возвращает объект класса <code>Query</code> , связанный с новым объектом класса <code>AndQuery</code> , который содержит объекты <code>q1</code> и <code>q2</code>
q1 q2	Возвращает объект класса <code>Query</code> , связанный с новым объектом класса <code>OrQuery</code> , содержащим объекты <code>q1</code> и <code>q2</code>
~q	Возвращает объект класса <code>Query</code> , связанный с новым объектом класса <code>NotQuery</code> , содержащим объект <code>q</code>
Классы реализации программы запросов	
Query_base	Абстрактный класс, базовый для классов запроса
WordQuery	Класс, производный от класса <code>Query_base</code> , который ищет указанное слово
NotQuery	Класс, производный от класса <code>Query_base</code> , представляющий набор строк, в которых указанный операнд <code>Query</code> отсутствует
BinaryQuery	Абстрактный базовый класс, производный от класса <code>Query_base</code> , который представляет запросы с двумя операндами типа <code>Query</code>
OrQuery	Класс, производный от класса <code>BinaryQuery</code> , который возвращает набор номеров строк, в которых присутствует хотя бы один из операндов
AndQuery	Класс, производный от класса <code>BinaryQuery</code> , который возвращает набор номеров строк, в которых присутствуют оба операнда

Упражнения раздела 15.9.1

Упражнение 15.31. При условии, что `s1`, `s2`, `s3` и `s4` являются строками, укажите, какие объекты создаются в следующих выражениях:

- (a) `Query(s1) | Query(s2) & ~ Query(s3);`
- (b) `Query(s1) | (Query(s2) & ~ Query(s3));`
- (c) `(Query(s1) & (Query(s2)) | (Query(s3) & Query(s4)));`

15.9.2. Классы `Query_base` и `Query`

Начнем реализацию с определения класса `Query_base`:

```
// абстрактный класс, являющийся базовым для конкретных типов запроса;
// все члены закрыты
class Query_base {
    friend class Query;
protected:
    using line_no = TextQuery::line_no; // используется в функциях eval()
    virtual ~Query_base() = default;
private:
    // eval() возвращает соответствующий запросу QueryResult
    virtual QueryResult eval(const TextQuery&) const = 0;
    // rep() строковое представление запроса
    virtual std::string rep() const = 0;
};
```

Обе функции, `eval()` и `rep()`, являются чистыми виртуальными, что делает класс `Query_base` абстрактным базовым (см. раздел 15.4, стр. 771). Поскольку класс `Query_base` не предназначен для пользователей и непосредственного использования в производных классах, у него нет открытых членов. Класс `Query_base` будет использоваться только через объекты класса `Query`. Класс предоставляет дружественные отношения классу `Query`, поскольку его члены вызывают виртуальные функции класса `Query_base`.

Защищенный член `line_no` будет использоваться в функциях `eval()`. Деструктор также будет защищен, поскольку он используется (неявно) деструкторами в производных классах.

Класс `Query`

Класс `Query` предоставляет интерфейс к иерархии наследования `Query_base` и скрывает ее. Каждый объект класса `Query` содержит указатель `shared_ptr` на соответствующий объект класса `Query_base`. Поскольку класс `Query` — единственный интерфейс к классам иерархии `Query_base`, он должен определить собственные версии функций `eval()` и `rep()`.

Конструктор `Query()`, получающий строку, создаст новый объект класса `WordQuery` и свяжет его указатель-член `shared_ptr` с этим недавно созданным объектом. Операторы `&`, `|` и `~` создают объекты `AndQuery`, `OrQuery` и `NotQuery` соответственно. Эти операторы возвращают объект класса `Query`, связанный с созданным им объектом. Для поддержки этих операторов класс `Query` нуждается в конструкторе, получающем указатель `shared_ptr` на класс `Query_base` и сохраняющем его. Сделаем этот конструктор закрытым, поскольку объекты класса `Query_base` не предназначены для определения общим пользовательским кодом. Так как этот конструктор является закрытым, операторы следует сделать дружественными.

Исходя из приведенного выше проекта, сам класс `Query` довольно прост:

```
// класс интерфейса для взаимодействия с иерархией
// наследования Query_base
class Query {
    // эти операторы должны обращаться к указателю .shared_ptr
    friend Query operator~(const Query &);
    friend Query operator|(const Query&, const Query&);
    friend Query operator&(const Query&, const Query&);
public:
    Query(const std::string&); // создает новый WordQuery
    // функции интерфейса: вызывают соответствующий оператор Query_base
    QueryResult eval(const TextQuery &t) const
        { return q->eval(t); }
    std::string rep() const { return q->rep(); }
private:
    Query(std::shared_ptr<Query_base> query): q(query) { }
    std::shared_ptr<Query_base> q;
};
```

Начнем с объявления дружественных операторов, создающих объекты класса `Query`. Эти операторы должны быть друзьями, чтобы использовать закрытый конструктор.

В открытом интерфейсе для класса `Query` объявляется, но еще не может быть определен получающий строку конструктор. Этот конструктор создает объект класса `WordQuery`, поэтому невозможно определить этот конструктор, пока не определен сам класс `WordQuery`.

Два других открытых члена представляют интерфейс для класса `Query_base`. В каждом случае оператор класса `Query` использует свой указатель класса `Query_base` для вызова соответствующей (виртуальной) функции класса `Query_base`. Фактически вызываемая версия определяется во время выполнения и будет зависеть от типа объекта, на который указывает указатель `q`.



Оператор вывода класса `Query`

Оператор вывода — хороший пример того, как работает вся система запросов:

```
std::ostream &
operator<<(std::ostream &os, const Query &query)
{
    // Query::rep() осуществляет виртуальный вызов через свой
    // указатель Query_base на rep()
    return os << query.rep();
}
```

При выводе объекта класса `Query` оператор вывода вызывает (открытую) функцию-член `rep()` класса `Query`. Эта функция осуществляет виртуальный вызов через свой указатель-член функции-члена `rep()` объекта, на который указывает данный объект класса `Query`.

```
Query andq = Query(sought1) & Query(sought2);
cout << andq << endl;
```

Таким образом, когда в коде встречается оператор вывода, он вызывает функцию `Query::rep()` объекта `andq`. Функция `Query::rep()` в свою очередь осуществляет виртуальный вызов через свой указатель класса `Query_base` на версию функции `rep()` класса `Query_base`. Поскольку объект `andq` указывает на объект класса `AndQuery`, этот вызов выполнит функцию `AndQuery::rep()`.

Упражнения раздела 15.9.2

Упражнение 15.32. Что будет при копировании, перемещении, присвоении и удалении объекта класса `Query`?

Упражнение 15.33. А объектов класса `Query_base`?

15.9.3. Производные классы

Самая интересная часть классов, производных от класса `Query_base`, в том, как они представляются. Класс `WordQuery` проще всех. Его задача — хранение искомого слова.

Другие классы работают на одном или двух операндах. У класса `NotQuery` один операнд, а у классов `AndQuery` и `OrQuery` — по два. Операндами в каждом из этих классов могут быть объекты любого из реальных классов, производных от класса `Query_base`: `NotQuery` может быть применен к `WordQuery`, как и `AndQuery`, `OrQuery` или `NotQuery`. Для обеспечения такой гибкости операнды следует хранить как указатели на класс `Query_base`. Таким образом, можно привязать указатель на любой необходимый реальный класс.

Но вместо того, чтобы хранить указатель на класс `Query_base`, классы будут сами использовать объект `Query`. Подобно тому, как пользовательский код упрощается при использовании класса интерфейса, можно упростить код собственного класса, используя тот же класс.

Теперь, когда конструкция этих классов известна, их можно реализовать.

Класс `WordQuery`

Класс `WordQuery` отвечает за поиск заданной строки. Это единственная операция, которая фактически выполняет запрос для данного объекта класса `TextQuery`:

```
class WordQuery: public Query_base {
    friend class Query; // Query использует конструктор WordQuery
    WordQuery(const std::string &s): query_word(s) { }
    // конкретный класс: WordQuery определяет все унаследованные чистые
    // виртуальные функции
```

```

QueryResult eval(const TextQuery &t) const
    { return t.query(query_word); }
std::string rep() const { return query_word; }
std::string query_word; // искомое слово
};

```

Подобно классу `Query_base`, у класса `WordQuery` нет открытых членов; он должен сделать класс `Query` дружественным, чтобы позволить ему получать доступ к конструктору `WordQuery()`.

Каждый из конкретных классов запроса должен определить унаследованные чистые виртуальные функции `eval()` и `rep()`. Обе функции определены в теле класса `WordQuery`: функция `eval()` вызывает функцию-член `query()` своего параметра типа `TextQuery`, который фактически осуществляет поиск в файле; функция `rep()` возвращает строку, которую данный объект класса `WordQuery` представляет (т.е. `query_word`).

Определив класс `WordQuery`, можно определить конструктор `Query()`, получающий строку:

```

inline
Query::Query(const std::string &s): q(new WordQuery(s)) { }

```

Этот конструктор резервирует объект класса `WordQuery` и инициализирует его указатель-член так, чтобы он указывал на этот недавно созданный объект.

Класс `NotQuery` и оператор `~`

Оператор `~` подразумевает создание объекта класса `NotQuery`, содержащего инверсный запрос:

```

class NotQuery: public Query_base {
    friend Query operator~(const Query &);
    NotQuery(const Query &q): query(q) { }
    // конкретный класс: NotQuery определяет все унаследованные
    // чистые виртуальные функции
    std::string rep() const {return "~(" + query.rep() + ")";}
    QueryResult eval(const TextQuery&) const;
    Query query;
};
inline Query operator~(const Query &operand)
{
    return std::shared_ptr<Query_base>(new NotQuery(operand));
}

```

Поскольку все члены класса `NotQuery` являются закрытыми, объявляем оператор `~` дружественным. Чтобы отобразить объект класса `NotQuery`, следует вывести символ `"~"`, сопровождаемый основным запросом. Чтобы сделать приоритет очевидным для читателя, заключим запрос в скобки.

Следует заметить то, что вызов функции `rep()` объекта класса `NotQuery` в конечном счете приводит к виртуальному вызову функции собственной функции-члена `rep(): query.rep()` — это не виртуальный вызов функции-члена

`rep()` класса `Query`. Функция `Query::rep()` в свою очередь осуществляет вызов `q->rep()`, являющийся виртуальным вызовом через указатель `Query_base`.

Оператор `~` динамически резервирует новый объект класса `NotQuery`. Оператор `return` (неявно) использует конструктор `Query()`, получающий указатель `shared_ptr<Query_base>`. Таким образом, оператор `return` эквивалентен следующему:

```
// резервировать новый объект NotQuery
// связать новый объект NotQuery с указателем shared_ptr<Query_base>
shared_ptr<Query_base> tmp(new NotQuery(expr));
return Query(tmp); // использовать конструктор Query(), получающий
// указатель shared_ptr
```

Функция-член `eval()` достаточно сложна, поэтому реализуем ее вне тела класса. Более подробно функция `eval()` рассматривается в разделе 15.9.4 (стр. 815).

Класс `BinaryQuery`

Класс `BinaryQuery` — это абстрактный базовый класс, содержащий данные, необходимые двум классам запроса, `AndQuery` и `OrQuery`, которые используют по два операнда:

```
class BinaryQuery: public Query_base {
protected:
    BinaryQuery(const Query &l, const Query &r, std::string s):
        lhs(l), rhs(r), opSym(s) { }
    // абстрактный класс: BinaryQuery функцию eval() не определяет
    std::string rep() const { return "(" + lhs.rep() + " "
                                   + opSym + " "
                                   + rhs.rep() + ")"; }

    Query lhs, rhs; // правый и левый операнды
    std::string opSym; // имя оператора
};
```

Данными класса `BinaryQuery` являются два операнда запроса и символ оператора. Конструктор получает эти два операнда и символ оператора, каждый из которых он хранит в соответствующих переменных-членах.

Чтобы отобразить объект класса `BinaryOperator`, следует вывести выражение в скобках, состоящее из левого операнда, оператора и правого операнда. Как и в случае класса `NotQuery`, вызов функции `rep()` в конечном счете осуществляет вызов виртуальных функций `rep()` объектов класса `Query_base`, на которые указывают параметры `lhs` и `rhs`.



Класс `BinaryQuery` не переопределяет функцию `eval()`, а следовательно, наследует ее чистой виртуальной. Таким образом, класс `BinaryQuery` остается абстрактным и его объекты создавать нельзя.

Классы `AndQuery`, `OrQuery` и их операторы

Классы `AndQuery` и `OrQuery`, а также соответствующие им операторы очень похожи:

```
class AndQuery: public BinaryQuery {
    friend Query operator&(const Query&, const Query&);
    AndQuery(const Query &left, const Query &right):
        BinaryQuery(left, right, "&") { }
    // конкретный класс: AndQuery наследует функцию rep(),
    // а остальные чистые виртуальные функции переопределяет
    QueryResult eval(const TextQuery&) const;
};
inline Query operator&(const Query &lhs, const Query &rhs)
{
    return std::shared_ptr<Query_base>(new AndQuery(lhs, rhs));
}

class OrQuery: public BinaryQuery {
    friend Query operator|(const Query&, const Query&);
    OrQuery(const Query &left, const Query &right):
        BinaryQuery(left, right, "|") { }
    QueryResult eval(const TextQuery&) const;
};
inline Query operator|(const Query &lhs, const Query &rhs)
{
    return std::shared_ptr<Query_base>(new OrQuery(lhs, rhs));
}
```

Эти классы объявляют соответствующий оператор дружественным и определяют конструктор, создающий их базовую часть класса `BinaryQuery` с соответствующим оператором. Они наследуют определение функции `rep()` от класса `BinaryQuery`, но каждый из них определяет собственную версию функции `eval()`.

Как и оператор `~`, операторы `&` и `|` возвращают указатель `shared_ptr` на вновь созданный объект соответствующего типа. Этот указатель `shared_ptr` приводится к типу `Query` в операторе `return` каждого из этих операторов.

Упражнения раздела 15.9.3

Упражнение 15.34. Исходя из выражения, представленного на рис. 15.3 (стр. 805):

- (a) Перечислите конструкторы, задействованные при обработке этого выражения;
- (b) Перечислите обращения к функции `rep()` из выражения `cout << q;`
- (c) Перечислите обращения к функции `eval()` из выражения `q.eval.`

Упражнение 15.35. Реализуйте классы `Query` и `Query_base`, включая определение функции `rep()`, но исключая определение функции `eval()`.

Упражнение 15.36. Добавьте операторы вывода в конструкторы и функции-члены `rep()`. Запустите код на выполнение, чтобы проверить свои ответы на вопросы (а) и (б) первого упражнения.

Упражнение 15.37. Какие изменения следовало бы внести в классы, если бы у производных классов были члены типа `shared_ptr<Query_base>`, а не типа `Query`?

Упражнение 15.38. Допустимы ли следующие объявления? Если нет, то почему? Если да, то что они означают?

```
BinaryQuery a = Query("fiery") & Query("bird");
AndQuery b = Query("fiery") & Query("bird");
OrQuery c = Query("fiery") & Query("bird");
```

15.9.4. Виртуальные функции `eval()`

Функции `eval()` — основа системы запросов. Каждая из них вызывает функцию `eval()` своего операнда (операндов), а затем применяет собственную логику вычислений: функция `eval()` класса `OrQuery` возвращает объединение результатов своих операндов, а функция `eval()` класса `AndQuery` возвращает их пересечение. Функция `eval()` класса `NotQuery` немного сложнее: она должна возвращать номера строк, не входящих в набор операнда.

Для обеспечения обработки в функциях `eval()` необходимо использовать ту версию класса `QueryResult`, в который определены члены, добавленные в упражнениях раздела 12.3.2 (стр. 624). Подразумевается, что у класса `QueryResult` есть функции-члены `begin()` и `end()`, позволяющие перебрать набор номеров строк, которые содержит объект класса `QueryResult`. Подразумевается также, что у класса `QueryResult` есть функция-член `get_file()`, возвращающая указатель `shared_ptr` на файл, к которому осуществляется запрос.



ВНИМАНИЕ

Класс `Query` использует функции-члены `begin()` и `end()`, определенные для класса `QueryResult` в упражнении 12.3.2 (стр. 624).

Функция `OrQuery::eval()`

Функция `eval()` класса `OrQuery` объединяет наборы номеров строк, возвращенных его операндами, т.е. ее результатом является объединение результатов двух операндов.

Объект класса `OrQuery` представляет объединение результатов двух своих операндов, полученных при вызове функции-члена `eval()` каждого из них. Поскольку эти операнды являются объектами класса `Query`, вызов функции `eval()` является вызовом `Query::eval()`, который в свою очередь осуществляет виртуальный вызов функции `eval()` объекта базового класса `Query_base`. Каждый из этих вызовов возвращает объект класса `QueryResult`, представляющий номера строк, в которых присутствует его операнд. Эти номера строк объединяются в новый набор:

```
// возвращает объединение наборов результатов своих операндов
QueryResult
OrQuery::eval(const TextQuery& text) const
{
    // виртуальные вызовы через члены Query, lhs и rhs
    // вызовы eval() возвращают QueryResult для каждого операнда
    auto right = rhs.eval(text), left = lhs.eval(text);
    // копировать номера строк левого операнда в результирующий набор
    auto ret_lines =
        make_shared<set<line_no>>(left.begin(), left.end());
    // вставить строки из правого операнда
    ret_lines->insert(right.begin(), right.end());
    // вернуть новый QueryResult, представляющий объединение lhs и rhs
    return QueryResult(rep(), ret_lines, left.get_file());
}
```

Набор `ret_lines` инициализируется с использования того конструктора, который получает пару итераторов. Функции-члены `begin()` и `end()` класса `QueryResult` возвращают итераторы на номера строк набора. Таким образом, набор `ret_lines` создается при копировании элементов из набора `left`. Затем для вставки элементов из набора `right` вызывается функция `insert()`. После этого вызова набор `ret_lines` содержит номера строк из наборов, которые присутствуют в наборах `left` или `right`.

Функция `eval()` завершает работу, создавая и возвращая объект класса `QueryResult`, представляющий объединение соответствий. Конструктор `QueryResult()` (см. раздел 12.3.2, стр. 622) получает три аргумента: строку, представляющую запрос, указатель `shared_ptr` на набор соответствующих номеров строк и указатель `shared_ptr` на вектор, представляющий входной файл. Вызов функции `rep()` позволяет создать строку, а вызов функции `get_file()` — получить указатель `shared_ptr` на файл. Поскольку оба набора, `left` и `right`, относятся к тому же файлу, не имеет значения, который из них использовать для функции `get_file()`.

Функция `AndQuery::eval()`

Версия функции `eval()` класса `AndQuery` подобна версии класса `OrQuery`, за исключением того, что она использует библиотечный алгоритм для поиска строк, общих для обоих запросов:

```
// возвращает пересечение наборов результатов своих операндов
QueryResult
AndQuery::eval(const TextQuery& text) const
{
    // виртуальный вызов через операнды класса Query для получения
    // результирующих наборов для операндов
    auto left = lhs.eval(text), right = rhs.eval(text);
    // набор для хранения пересечения left и right
    auto ret_lines = make_shared<set<line_no>>();
    // выводит пересечение двух диапазонов в итератор назначения
    // итератор назначения в этом вызове добавляет элементы в ret
}
```



```

    set_intersection(left.begin(), left.end(),
                    right.begin(), right.end(),
                    inserter(*ret_lines, ret_lines->begin()));
    return QueryResult(rep(), ret_lines, left.get_file());
}

```

Здесь для объединения двух наборов используется библиотечный алгоритм `set_intersection`, описанный в приложении А.2.8 (стр. 1094).

Алгоритм `set_intersection` получает пять итераторов. Первые четыре он использует для обозначения двух исходных последовательностей (см. раздел 10.5.2, стр. 529). Его последний аргумент обозначает получателя. Алгоритм выводит элементы, присутствующие в обоих исходных последовательностях, в результирующую.

В данном вызове получателем является итератор вставки (см. раздел 10.4.1, стр. 514). Результатом записи алгоритмом `set_intersection` в этот итератор будет вставка нового элемента в набор `ret_lines`.

Подобно функции `eval()` класса `OrQuery`, эта завершается созданием и возвращением объекта класса `QueryResult`, представляющего объединение соответствий.

Функция `NotQuery::eval()`

Функция `eval()` класса `NotQuery` ищет в тексте все строки, в которых операнд отсутствует.

```

// возвращает строки, отсутствующие в наборе результатов операнда
QueryResult
NotQuery::eval(const TextQuery& text) const
{
    // виртуальный вызов для вычисления операнда Query
    auto result = query.eval(text);
    // начать с пустого результирующего набора данных
    auto ret_lines = make_shared<set<line_no>>();
    // следует перебрать строки, в которых присутствует операнд
    auto beg = result.begin(), end = result.end();
    // для каждой строки во входном файле, если она отсутствует
    // в result, добавить ее номер в ret_lines
    auto sz = result.get_file()->size();
    for (size_t n = 0; n != sz; ++n) {
        // если не обработаны все строки в result
        // проверить присутствие этой строки
        if (beg == end || *beg != n)
            ret_lines->insert(n); // если нет в result, добавить строку
        else if (beg != end)
            ++beg; // в противном случае получить следующий номер строки
        // в result, если она есть
    }
    return QueryResult(rep(), ret_lines, result.get_file());
}

```

Как и другие функции `eval()`, данная начинается с вызова функции `eval()` операнда объекта. Этот вызов возвращает объект класса `QueryResult`, содержа-

ций номера строк, в которых присутствует операнд. Однако вернуть необходимо набор номеров строк, в которых операнд отсутствует. Как и в других функциях `eval()`, данная начинается с вызова функции `eval()` операнда объекта. Вызов возвращает объект класса `QueryResult`, содержащий номера строк, в которых операнд присутствует, но необходимы номера строки, на которых операнд отсутствует. Поэтому следует найти в файле все строки, отсутствующие в наборе результатов.

Набор создается в результате последовательного перебора целых чисел до размера входного файла. Каждое число, отсутствующее в наборе `result`, помещается в набор `ret_lines`. Итераторы `beg` и `end` устанавливаются на первый и следующий после последнего элементы в наборе `result`. Поскольку речь идет о наборе, при переборе номера строк будут следовать в порядке возрастания.

Тело цикла проверяет наличие текущего числа в наборе `result`. Если его нет, то число добавляется в набор `ret_lines`. Если он есть, осуществляется приращение итератора `beg` набора `result`.

Как только все номера строк будут обработаны, возвращается объект класса `QueryResult`, содержащий набор `ret_lines` наряду с результатами выполнения функций `rep()` и `get_file()`, как и у предыдущих функций `eval()`.

Упражнения раздела 15.9.4

Упражнение 15.39. Реализуйте классы `Query` и `Query_base`. Проверьте приложение на вычислении и выводе запроса, представленного на рис. 15.3 (стр. 805).

Упражнение 15.40. Что будет, если параметр `rhs` функции-члена `eval()` класса `OrQuery` возвратит пустой набор? Что, если так поступит ее параметр `lhs`? Что если и `rhs`, и `lhs` возвратят пустые множества?

Упражнение 15.41. Переделайте свои классы так, чтобы использовать встроенные указатели на класс `Query_base`, а не интеллектуальные указатели `shared_ptr`. Помните, что ваши классы больше не смогут использовать синтезируемые функции-члены управления копированием.

Упражнение 15.42. Разработайте и реализуйте одно из следующих дополнений.

- (a) Организуйте вывод слов только однажды в предложении, а не однажды в строке.
- (b) Снабдите систему историей, позволяющей пользователю обратиться к предыдущему запросу по номеру, а также добавлять или комбинировать их с другими.
- (c) Позвольте пользователю ограничивать результаты так, чтобы отображался набор соответствий только в заданном диапазоне строк.

Резюме

Наследование позволяет создавать новые классы, которые совместно используют возможности их базового класса (классов), но при необходимости могут их переопределить или дополнить. Динамическое связывание позволяет компилятору во время выполнения выбрать версию применяемой функции на основании динамического типа объекта. Комбинация наследования и динамического связывания позволяет создавать программы, которые либо не зависят от типа объекта, либо имеют поведение, зависящее от типа объекта.

В языке C++ динамическое связывание применимо *только* к тем функциям, которые объявлены виртуальными и вызываются при помощи ссылок или указателей.

Объекты производных классов состоят из части (частей) базового класса и части производного. Поскольку частью объекта производного класса является объект базового, ссылку или указатель на объект производного класса вполне можно преобразовать в ссылку или указатель на его доступный базовый класс.

При создании, копировании, перемещении и присвоении объектов производного класса сначала создается, копируется, перемещается и присваивается базовая часть объекта. Деструкторы выполняются в обратном порядке: сначала удаляется производная часть, затем выполняются деструкторы частей базовых классов. Базовые классы обычно определяют виртуальный деструктор, даже если у них нет никакой потребности в деструкторе.

Производный класс определяет уровень защиты для каждого из своих базовых классов. Члены открытого базового класса являются частью интерфейса производного класса; члены закрытого базового класса недоступны; члены защищенного базового класса доступны для классов, производных от него, но не для пользователей производного класса.

Термины

Абстрактный класс (abstract base class). Класс, обладающий одной или несколькими чистыми виртуальными функциями. Нельзя создать объекты типа абстрактного базового класса.

Базовый класс (base class). Класс, от которого происходит другой класс. Члены базового класса становятся членами производного класса.

Виртуальная функция (virtual function). Функция-член, обеспечивающая зависимое от типа поведение. Во время выполнения выбор конкретной версии функции при обращении к виртуальной функции с помощью ссылки или указателя осуществляется на основании типа объекта, с которым связана ссылка или указатель.

Динамический тип (dynamic type). Тип объекта во время выполнения. Динамический тип объекта, на который ссылается ссылка или указывает указатель, может отличаться

ся от статического типа ссылки или указателя. Указатель или ссылка на тип базового класса может применяться к объекту производного типа. В таких случаях статическим типом будет ссылка (или указатель) на базовый класс, а динамическим — ссылка (или указатель) на производный.

Динамическое связывание (dynamic binding). Отсрочка выбора выполняемой функции до времени выполнения. В языке C++ динамическим связыванием называют выбор во время выполнения используемой версии виртуальной функции на основании фактического типа объекта, который связан со ссылкой или с указателем.

Доступность (accessible). Член базового класса доступен через производный объект. Доступность зависит от спецификатора доступа, используемого в списке наследования производного класса, и уровня доступа члена в базовом классе. Например, открытый (public) член класса, унаследованный при открытом наследовании, доступен для пользователей производного класса. Открытый член базового класса недоступен, если наследование является закрытым.

Закрытое наследование (private inheritance). При закрытом наследовании открытые и защищенные члены базового класса становятся закрытыми членами производного.

Защищенное наследование (protected inheritance). При защищенном наследовании защищенные и открытые члены базового класса становятся защищенными членами производного.

Косвенный базовый класс (indirect base class). Базовый класс, отсутствующий в списке наследования производного класса. Класс, от которого наследуется прямой базовый класс, прямо или косвенно является косвенным базовым классом для производного класса.

Наследование (inheritance). Программная технология определения нового класса (производного) в терминах существующего класса (базового). Производный класс наследует члены базового класса.

Объектно-ориентированное программирование (object-oriented programming). Техника программирования с использованием абстракции данных, наследования и динамического связывания.

Открытое наследование (public inheritance). Открытый интерфейс базового класса является частью открытого интерфейса производного класса.

Отсечение (sliced down). Происходящее при использовании объекта производного типа для инициализации или присвоения объекта базового типа. Производная часть объекта отсекается, оставляя только базовую часть, которая и присваивается объекту базового типа.

Переопределение (override). Виртуальная функция, определенная в производном классе, с тем же списком параметров, что и у виртуальной функции в базовом классе, переопределяет определение базового класса.

Полиморфизм (polymorphism). Применительно к объектно-ориентированному программированию — возможность получить специфическое для типа поведение на основании динамического типа ссылки или указателя.

Преобразование производного класса в базовый (derived-to-base conversion). Неявное преобразование объекта производного класса в ссылку на базовый класс или указатель на объект производного класса в указатель на базовый класс.

Привязка во время выполнения (run-time binding). С.м. динамическое связывание.

- Производный класс** (derived class). Класс, унаследованный от другого класса. Производный класс может переопределить виртуальные функции своего базового класса и определять новые члены. Область видимости производного класса вкладывается в область ее базового класса (классов); члены производного класса могут использовать члены базового класса непосредственно.
- Прямой базовый класс** (direct base class). Базовый класс, от которого непосредственно происходит производный. Прямые базовые классы определяются в списке наследования производного класса. Прямой базовый класс сам может быть производным классом.
- Рефакторинг** (refactoring). Способ перепроектирования программ, позволяющий собрать взаимосвязанные части в единую абстракцию при замене первоначального кода новой абстракцией. Рефакторинг классов, как правило, применяют для перемещения переменных или функций-членов в самый верхний общий пункт иерархии во избежание дублирования кода.
- Спецификатор доступа** `protected`. К членам, определенным после ключевого слова `protected`, могут обращаться только члены производного класса и друзья. Однако доступны эти члены только через производные объекты. Защищенные члены не доступны для обычных пользователей класса.
- Список наследования класса** (class derivation list). Список базовых классов, от которых происходит производный класс; у каждого из них может быть необязательный уровень доступа. Если спецификатора доступа нет, наследование открытое (`public`), если производный класс определен с ключевым словом `struct`, и закрытое (`private`), если класс определен с ключевым словом `class`.
- Статический тип** (static type). Тип, с которым определяется переменная или возвращает выражение. Статический тип известен во время компиляции.
- Чистая виртуальная функция** (pure virtual). Виртуальная функция, объявленная в заголовке класса с использованием `= 0` в конце списка параметров функции. Чистая виртуальная функция не обязана (но вполне может) быть определена классом. Класс с чистой виртуальной функцией является абстрактным. Если производный класс не определяет собственную версию унаследованной чистой виртуальной функции, он также становится абстрактным.

